Spatially Prompted Visual Trajectory Prediction for Egocentric Manipulation

Yifan Li[1]*, Xinyu Zhou[1]*, Yunhao Ge[2], Xiaobo Tan[1], Yu Kong[1]
[1]Michigan State University, [2]NVIDIA Research

Code Project Dataset

Abstract

Language instructions offer a flexible way to specify task semantics but can become ambiguous in cluttered scenes containing multiple visually similar objects and candidate targets. We study Spatially Prompted Visual Trajectory Prediction *(SP-VTP), an open-loop setting in which points or bounding boxes in the first egocentric frame specify an object–target pair and condition the prediction of future 3D end-effector (EE) trajectory chunks. To support this setting, we introduce* EgoSPT*, a controlled dataset comprising 2,841 egocentric fork pick-and-place episodes with first-frame object and target annotations and reconstructed 3D EE trajectories. SP-VTP evaluates whether a static task specification remains informative as the viewpoint and scene configuration evolve throughout execution. We also introduce* SPOT*, a spatially prompted object–target trajectory predictor that integrates first-frame visual and coordinate prompts with current visual observations and motion history to generate future trajectories. Rather than relying on an explicit object tracker, SPOT maintains a fixed representation of the first-frame task specification. Across three training seeds, spatial prompting consistently reduces translation and endpoint errors relative to a no-prompt baseline, while controlled prompt interventions reveal distinct roles for object and target cues at different stages of execution. Together, EgoSPT and SPOT provide a dataset and baseline for spatially conditioned egocentric trajectory prediction in open-loop pick-and-place settings.*

1. Introduction

Most robot policies rely on language [19, 30, 40, 41, 60] or task identifiers [55, 56]. Language is expressive for task semantics, but can under-specify the intended instance or placement region in cluttered scenes with visually similar candidates. In such cases, a point or box prompt on the first egocentric frame provides a direct way to indicate *this* object and *that* target while preserving the visual context needed for action. We view spatial prompts as a complementary instance-grounding interface for language.

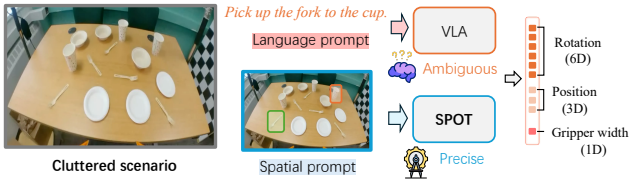

Figure 1. Motivation for SP-VTP. Language instructions can be ambiguous when multiple visually similar objects and candidate targets appear in a cluttered scene. SP-VTP uses explicit first-frame object and target prompts to condition future egocentric end-effector trajectory prediction.

We formulate this setting as *Spatially Prompted Visual Trajectory Prediction* (SP-VTP). Given first-frame points or boxes indicating the object and target, the model predicts future end-effector (EE) trajectory chunks from streaming egocentric observations (see Fig. 1). Unlike grounding or tracking, the output is not a mask, box, or object identity, but a sequence of relative 6D EE motions and gripper states. SP-VTP is therefore an open-loop trajectory-prediction problem that tests whether a sparse initial task specification remains useful as the visual context evolves.

Recent hierarchical VLA systems also use spatial representations or planning intermediates for manipulation [23, 48, 50–52, 54, 57]. For example, HAMSTER predicts coarse 2D image-plane paths from RGB observations and language instructions, which then guide a separate low-level 3D-aware controller [23]. In SP-VTP, a static first-frame object–target spatial prompt is given directly as the task specification, while the model predicts future relative 3D EE trajectory chunks from evolving egocentric observations. Language is omitted in this benchmark to isolate the role of spatial task specification. SP-VTP brings together several challenges. The task specification is static, whereas the camera moves, the end effector occludes the scene, and the object relocates after grasping. Clutter and same-category distractors make the initial specification consequential, and the same object–target pair requires different motions across

*Equal contribution. Correspondence to: liyifa11@msu.edu.

execution phases. SP-VTP evaluates whether the initial task specification remains useful as the egocentric view changes throughout execution.

To study this problem, we introduce *EgoSPT*, an egocentric dataset of spatially prompted manipulation trajectories collected with a modified Universal Manipulation Interface (UMI) [4]. EgoSPT provides first-frame object and target annotations, egocentric visual observations, recovered 3D EE trajectories, and a scene-stratified task-level split. It covers one manipulated-object category (forks) and one task (pick-and-place), with five fork instances and nine receptacle targets across three scenes. This controlled setting supports analysis of spatial prompting under layout and clutter variation; transfer to other object categories and manipulation tasks remains untested.

Building on EgoSPT, we introduce *SPOT*, a spatially prompted object–target trajectory predictor. First-frame prompts specify the task, current observations and motion history provide execution context, and a decoder-style flow-matching head predicts a coherent future EE trajectory chunk. SPOT represents bounding boxes both as markings rendered on the first frame and as coordinate prompt tokens. These tokens attend to first-frame visual features to form a fixed task representation that conditions predictions together with the evolving observation context; SPOT does not contain an explicit object tracker. Because egocentric backgrounds, camera motion, and object layouts are strongly correlated, sample-level random splits can overestimate performance. We therefore keep all episodes from a selected scene–task group within one partition while retaining all three scenes in both training and validation. Per-scene results characterize layout complexity rather than cross-scene generalization. We report continuous translation, rotation, and gripper errors together with thresholded endpoint criteria; all are open-loop prediction metrics rather than robot task success. Our contributions are fourfold:

- We define *SP-VTP*, an open-loop setting for 3D visual trajectory prediction, in which object–target prompts specified in the first frame condition the prediction of future relative 3D EE trajectory chunks from evolving egocentric observations.
- We introduce *EgoSPT*, a controlled egocentric dataset for spatially prompted trajectory prediction that provides object–target annotations, reconstructed 3D EE trajectories, and a comprehensive evaluation protocol for predicted trajectories.
- We propose *SPOT*, a trajectory-prediction baseline that integrates visual and coordinate prompts with current observations and trajectory history, without relying on an explicit object tracker.
- We provide multi-seed statistics, comparisons with external VLA baselines, various prompting paradigms, and controlled prompt and execution-phase analyses.

2. Related Work

Goal-conditioned Robot Policies for Manipulation. Goal-conditioned manipulation policies generate actions from sensor observations under task specifications such as identifiers, goal images, or language instructions. Task identifiers compactly encode discrete intents but do not ground the object instance or target region in the current scene [55, 56]. Goal images specify the desired final state but require an example of the completed scene and may entangle task intent with background or object layout [29, 34, 38, 43].
Language offers a flexible interface [8, 16, 30, 40, 41], and generalist VLA models extend language-conditioned control across tasks and embodiments [3, 19, 36, 60]. However, instructions can remain ambiguous when several visually similar objects and candidate targets appear together. VIMA [17] interleaves language and images in task prompts for autoregressive robot manipulation in simulation. Spatial alternatives include KITE, which grounds language into 2D keypoints for downstream manipulation, and RT-Trajectory, which conditions visuomotor policies on coarse trajectory sketches [11, 44]. VP-VLA uses structured visual prompts between high-level planning and low-level control [50]. Unlike VIMA's general multimodal task interface, SP-VTP uses static first-frame object–target points or boxes to predict relative 3D EE trajectory chunks from evolving egocentric observations.
Egocentric Manipulation Datasets. Human-centered datasets capture wearable-camera video for activity understanding [6, 9, 10], 3D hand–object interaction [1, 21, 46], and imitation learning from human video [18]. Manipulator-centered demonstration datasets place cameras on or near grippers or robotic hands, aligning the visual stream with manipulation execution and the action trajectories used for policy learning. UMI introduces a portable hand-held interface for collecting in-the-wild demonstrations and learning deployable visuomotor policies [4]. Subsequent UMI-style systems extend this action-aligned collection paradigm to diverse practical settings [12, 13, 24, 26, 28, 33, 39, 47, 53, 59]. Building on a modified UMI setup, EgoSPT pairs egocentric videos and recovered EE trajectories with first-frame object and target annotations, making the task specification explicit for spatially prompted trajectory prediction.
Egocentric Visual Trajectory Prediction. Human-centered forecasting predicts image-space hand motion and contact regions [14, 27, 31], or 3D action targets and hand trajectories from egocentric RGB video [2, 7, 22]. Manipulator-centered imitation learning instead represents actions as future EE pose and gripper-state chunks [5, 58], which UMI-style systems learn from egocentric demonstrations [4, 13, 33]. SP-VTP focuses on spatial task conditioning: an initial object–target prompt specifies what to manipulate and where to place it, while subsequent observations condition relative EE trajectory prediction.

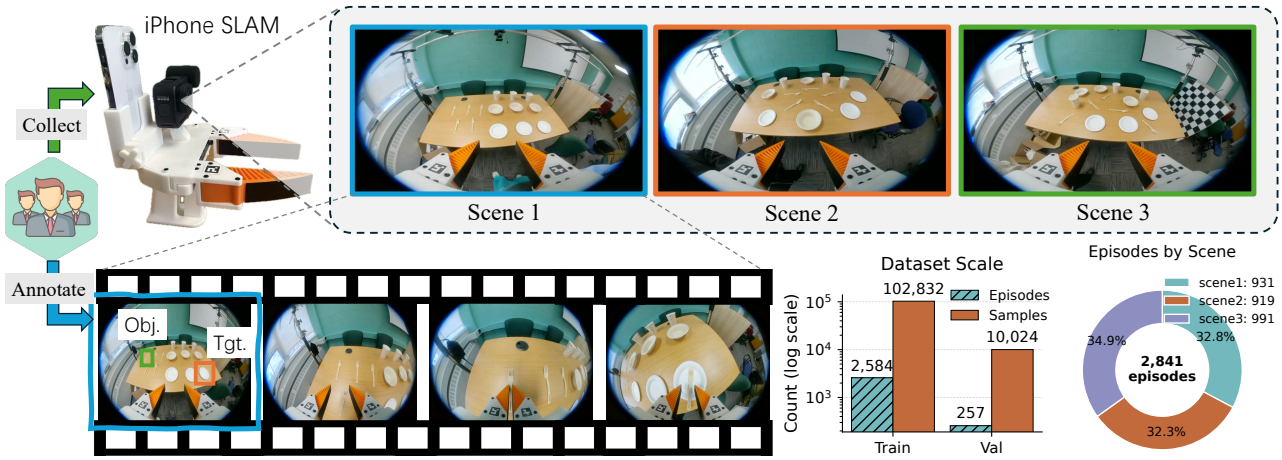

Figure 2. **Illustration of the EgoSPT dataset.** We use a modified UMI device equipped with an iPhone and a GoPro to collect EgoSPT, an egocentric visual trajectory dataset containing five forks and nine targets, including three plates, three bowls, and three cups. EgoSPT covers three scenes designed to evaluate different policy capabilities.

3. EgoSPT

EgoSPT evaluates whether first-frame spatial prompts can be translated into future egocentric trajectories. As shown in Fig. 2, it contains 2,841 pick-and-place videos with egocentric observations and recovered end-effector (EE) trajectories, collected using a modified UMI device [4, 12]. The device integrates a GoPro with a 170° fisheye lens for egocentric video capture and an iPhone for 6-DoF EE trajectory tracking. Leveraging the iPhone's visual–inertial SLAM system, the modified UMI provides more stable trajectory recovery than the original one. Nine trained experts collected and annotated the dataset, where each episode involves picking one of five visually similar forks and placing it into one of nine targets (three cups, three bowls, and three plates).

EgoSPT is organized into three scenes with increasing difficulty. Scene 1 contains structured layouts with 45 object–target combinations and 20 episodes per combination, testing object–target association under clean conditions. Scene 2 uses a cluttered layout with the same 45 combinations and 20 episodes per combination, evaluating robustness to distractors and spatial ambiguity. Scene 3 contains 22 randomly cluttered subscenes, each covering the 45 combinations with one episode per combination, enabling evaluation under diverse and low-data conditions. Together, these scenes form a progressive protocol for measuring behavior across structured and cluttered layouts. Because all three scenes occur in both training and validation, per-scene results are not used to claim cross-scene generalization.

Each video is approximately five seconds long at 30 fps and is downsampled to 10 fps, yielding about 50 trajectory steps per episode. Sliding-window training with horizon $H = 16$ produces roughly 110K trajectory prediction samples. Each episode provides a first frame, current frames, object/target spatial prompts, trajectory history, and future relative trajectory chunks. More statistic results can be found in the Appendix.

The dataset stresses several realistic factors: multiple same-category object instances, three target categories, fisheye egocentric observations, handheld camera motion, and clutter variation. These factors make it a testbed for spatial task conditioning, not merely trajectory regression.

4. SPOT

As shown in Fig. 3, SPOT is a spatial prompt conditioned trajectory policy composed of three modules: a task encoder, an observation encoder, and a trajectory generator. The task input combines visual prompts, obtained by rendering object and target boxes on the first frame, with coordinate prompts, which encode the same boxes as spatial tokens. The observation encoder represents the current egocentric frame and recent trajectory history. Conditioned on the resulting task and observation tokens, the trajectory generator predicts a future EE trajectory chunk.

Problem Formulation. SP-VTP takes a first-frame egocentric image I_0 with an object prompt p_{obj} and a target prompt p_{tgt}, where each prompt is either a point or a box in normalized image coordinates. At timestep t, the policy observes the current egocentric frame I_t and a raw history $H_t \in \mathbb{R}^{K\times 10}$ of the previous K actions. The goal is to predict a future trajectory chunk $A_t \in \mathbb{R}^{H\times 10} = \{a_t, \ldots, a_{t+H-1}\}, H = 16$. Each waypoint $a_{t+h} \in \mathbb{R}^{10}$ is parameterized as a 10D vector containing relative translation, 6D rotation, and gripper width. Let $T_t \in SE(3)$ denote the EE pose at timestep t as a homogeneous transformation matrix in the world frame: $T_t \in \mathbb{R}^{4\times 4} = \begin{bmatrix} R_t & \mathbf{p}_t \\ \mathbf{0}^\top & 1 \end{bmatrix}$, where

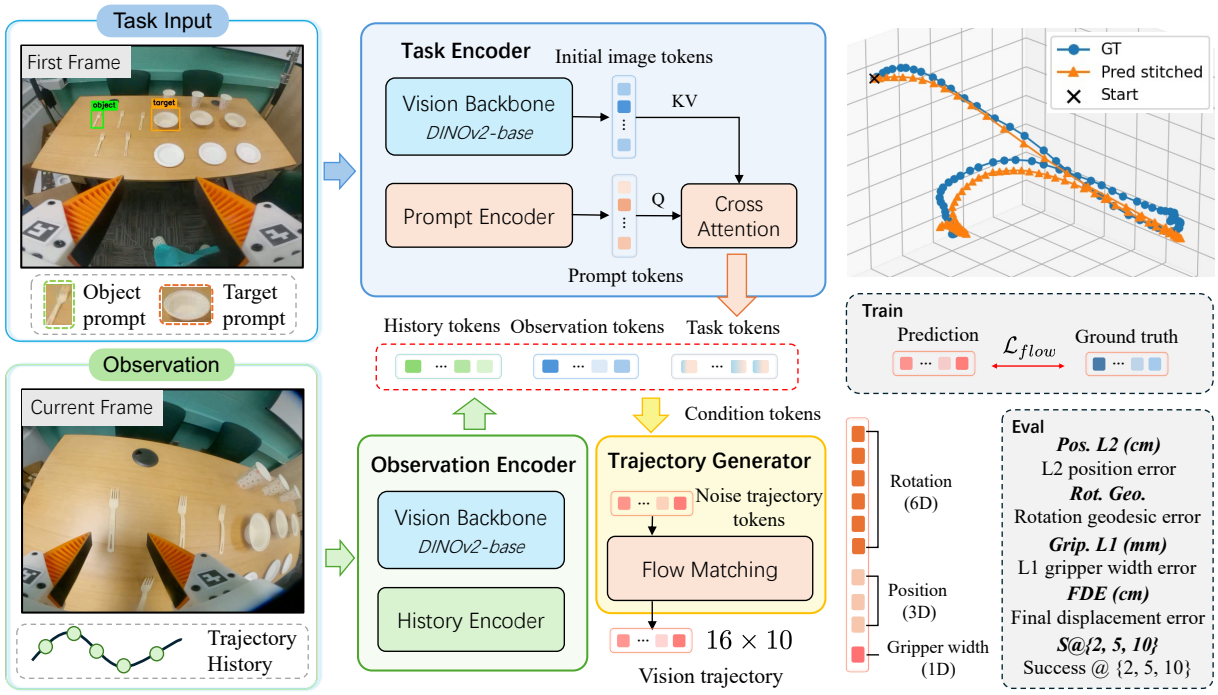

Figure 3. Overview of the proposed SPOT framework. Given a first-frame task specification, in which object and target boxes are rendered on the image and encoded as coordinate-prompt tokens, the task encoder extracts visual and prompt tokens and fuses them via cross-attention into a spatially prompted task representation. At each timestep, the observation encoder processes the current egocentric frame and action history, while the trajectory generator predicts future EE pose chunks using a flow-matching objective conditioned on the task and observation tokens. During training, the predicted trajectories are supervised by ground-truth trajectories using $\mathcal{L}_{\text{flow}}$. During evaluation, performance is measured using Pos. L2, Rot. Geo., Grip. L1, and FDE.

$R_t \in SO(3)$ is the EE orientation and $\mathbf{p}_t \in \mathbb{R}^3$ is its position. The relative pose target is computed in the current EE coordinate frame:

$$\Delta T_{t+h} = T_t^{-1} T_{t+h} = \begin{bmatrix} \Delta R_{t+h} & \Delta \mathbf{p}_{t+h} \\ \mathbf{0}^\top & 1 \end{bmatrix}, \quad h \in [1, H]. \tag{1}$$

Here, T_t^{-1} transforms future poses from the world frame into the coordinate frame attached to the current EE, so ΔT_{t+h} represents the rigid motion needed to move from the current pose to the future pose. We then convert this relative transform into the 10D waypoint target:

$$a_{t+h} = [\Delta \mathbf{p}_{t+h}, \text{rot6d}(\Delta R_{t+h}), g_{t+h}] \in \mathbb{R}^{10},$$

where $\Delta \mathbf{p}_{t+h} \in \mathbb{R}^3$ is the relative translation, $\text{rot6d}(\Delta R_{t+h}) \in \mathbb{R}^6$ is the relative rotation, and $g_{t+h} \in \mathbb{R}$ is the gripper width. This representation asks the model to predict where the EE should move next relative to its current state, rather than where it is in a global frame.

Task Encoder. The task encoder processes the first frame I_0 together with the object and target prompts $(p_{\text{obj}}, p_{\text{tgt}})$. In the default setting, p_{obj} and p_{tgt} are bounding boxes. We render these boxes on the first frame to obtain a visually prompted image $\tilde{I}_0$, and also keep their normalized box coordinates as coordinate prompts. A frozen DINOv2 ViT-B/14 [37] backbone extracts and projects the first-frame features to the policy dimension $D = 768$, yielding $F_0 = [z_{\text{img}}; F_0^{\text{patch}}] \in \mathbb{R}^{(N+1)\times D}$, where F_0^{patch} contains the N patch tokens and z_{img} is the image-summary token, typically the projected DINOv2 CLS token. Coordinate prompts are encoded geometrically. Each 2D prompt coordinate is mapped through Fourier positional features [32, 45] and an MLP (prompt encoder), and a learnable role embedding identifies whether the token belongs to the object or the target. A box prompt contributes two corner tokens for the object and two corner tokens for the target, while a point prompt contributes one object token and one target token. Let M denote the total number of coordinate-prompt tokens, so $M = 4$ for boxes and $M = 2$ for points. The encoded prompts $Z_{\text{prompt}} \in \mathbb{R}^{M\times D}$ query the first-frame visual tokens through a Transformer decoder [49]:

$$\begin{aligned} Z_{\text{fused}} &= \text{CrossAtt}(Q = Z_{\text{prompt}}, KV = F_0) \in \mathbb{R}^{M\times D}, \\ Z_{\text{task}} &= [z_{\text{img}}; Z_{\text{fused}}] \in \mathbb{R}^{(M+1)\times D}. \end{aligned} \tag{2}$$

This prompt-to-image cross-attention lets the object and target prompts actively read task-relevant visual information

from the first frame. Appending the image-summary token to the M fused prompt tokens gives a task representation containing both global scene context and grounded object/target evidence. In the default bounding-box setting, $M = 4$ and $Z_{\text{task}} \in \mathbb{R}^{5\times D}$.

Observation Encoder. The observation encoder represents the execution state at timestep t. The current frame I_t is encoded by the same frozen visual backbone and projection layer used by the task encoder, ensuring that first-frame task tokens and current-frame observation tokens live in a shared visual space. This shared visual encoding makes it easier for the trajectory generator to relate the original task specification to the current egocentric view.

To provide short-term motion context, a lightweight MLP, $\text{Linear}(10, D)$–GELU–$\text{Linear}(D, D)$, maps each raw action in H_t to dimension D; learnable temporal-position and history-type embeddings are then added. Denoting this history encoder by E_{hist} and the current-frame image tokens by $F_t \in \mathbb{R}^{(N+1)\times D}$, we obtain

$$Z_{\text{hist}} = E_{\text{hist}}(H_t) \in \mathbb{R}^{K\times D},$$
$$Z_{\text{obs}} = [F_t; Z_{\text{hist}}] \in \mathbb{R}^{(N+1+K)\times D}.$$

We set $K = 4$ by default, giving $Z_{\text{obs}} \in \mathbb{R}^{(N+5)\times D}$. If no trajectory history is used, the observation encoder returns only current-frame visual tokens. Otherwise, Z_{obs} jointly describes what the robot currently sees and how the EE has recently moved.

Trajectory Generator. The trajectory generator predicts future trajectory chunks from the encoded task and execution context. It conditions on the concatenated task and observation tokens, $Z_{\text{cond}} = [Z_{\text{task}}; Z_{\text{obs}}]$. In the default visual-prompt plus box-coordinate setting, Z_{task} consists of one image-summary token from the prompted first frame and four fused coordinate-prompt tokens, for five task tokens in total. For a 224×224 ViT-B/14 input, the current observation provides $N + 1 = 257$ image tokens, while the history horizon contributes four trajectory-history tokens. Thus, $Z_{\text{cond}} \in \mathbb{R}^{(N+10)\times D}$ contains 266 tokens in total, capturing the task specification, current egocentric context, and recent motion history.

SPOT supports both diffusion [5, 15] and flow-matching [25]trajectory heads, using the same transformer decoder-style architecture in both cases. Noisy or interpolated trajectory tokens serve as the decoder targets, and Z_{cond} serves as the cross-attention memory. Each future waypoint can self-attend to other predicted waypoints and cross-attend to the task, observation, and history tokens.

The default head uses flow matching. Let x_0 denote the normalized ground-truth trajectory chunk and $\epsilon \sim \mathcal{N}(0, I)$ denote Gaussian noise. We sample $t \in [0, 1]$ and construct a straight interpolation path

$$x_t = (1 - t)x_0 + t\epsilon. \tag{3}$$

The target velocity is

$$v^\star(x_t, t) = \epsilon - x_0. \tag{4}$$

The flow head predicts this velocity from interpolated trajectory tokens and the condition memory:

$$\mathcal{L}_{\text{flow}} = \mathbb{E}\left[\|v_\theta(x_t, t, Z_{\text{cond}}) - (\epsilon - x_0)\|_2^2\right]. \tag{5}$$

At inference time, SPOT starts from Gaussian noise and integrates the learned velocity field from noise to a trajectory chunk using a small number of Euler steps. Future trajectory chunks are normalized by dataset-level waypoint mean and standard deviation before training. The trajectory head predicts in this normalized space, and predictions are denormalized after sampling. This balances translation, rotation, and gripper dimensions during optimization. During inference, the generated $H \times 10$ trajectory chunk is denormalized and interpreted as relative EE motion.

5. Experiments

5.1. Experimental Setup

Protocol. All models train on Scenes 1–3 and share a held-out set of 257 episodes (10,024 sliding-window chunks); scene-wise results measure layout complexity, not cross-scene generalization. Unless noted, evaluation uses recorded history in independent windows; Section 5.6 separately tests sequential predicted-state feedback. Default SPOT uses rendered and coordinate object/target boxes, frozen DINOv2 ViT-B/14, cross-attention task fusion, $K = 4$ history, and a flow-matching head. Prompt baselines use seeds 42/0/1 with split seed 42; diagnostic interventions use seed 42 and fixed initial action noise unless noted.

Metrics. We report mean relative translation (Pos. L2), final displacement (FDE), rotation geodesic (Rot. geo.), and gripper-width L1 (Grip. L1) errors in centimeters, degrees, and millimeters, respectively. Rot. geo. averages the relative $\text{SO}(3)$ angle after mapping 6D rotations via Gram–Schmidt. Endpoint metrics include Success@Xcm ($X \in \{2, 5, 10\}$), gripper-state accuracy (G-end; 30 mm threshold), and Combined@5/10, which additionally requires rotation within 20° and the correct gripper state. We emphasize that these metrics evaluate offline trajectory prediction rather than closed-loop task success.

5.2. Comparison with External VLA Baselines

We compare SPOT against SmolVLA [42], OpenVLA-OFT [20], GR00T N1.6 [35], and π0.5 [3] using language-only prompts. These baselines receive current observation, an action history of $K = 4$, and category-level language prompts, without access to the first frame or object and target boxes. All reported results are aggregated across three random seeds over 10,024 samples.

Table 1. Three-seed external-VLA comparison without language instance IDs. Four VLA baselines receive current observation image, $K = 4$ history action chunks, and category language prompts without spatial prompts. We report mean ± sample standard deviation over seeds 42/0/1 on 10,024 samples, followed by scene-wise Pos. L2/FDE (cm) and S@10 (%).

Method	Pos. L2 (cm)	FDE (cm)	Rot. geo. (°)	Grip. L1 (mm)	S@2 (%)	S@5 (%)	S@10 (%)
π0.5 [3]	14.25±0.18	23.68±0.28	15.53±0.02	17.18±0.18	0.33±0.02	4.64±0.09	20.67±0.47
SmolVLA [42]	9.36±0.22	16.85±0.47	11.06±0.16	11.75±0.31	1.53±0.17	12.70±0.90	37.21±1.72
OpenVLA-OFT [20]	8.16±0.06	15.33±0.14	**8.86±0.05**	**8.00±0.15**	3.24±0.54	17.14±0.68	41.43±0.74
GR00T N1.6 [35]	10.48±1.07	17.39±1.18	11.60±0.47	11.69±1.65	0.88±0.29	9.42±2.64	32.42±4.47
SPOT	**6.90±0.10**	**11.14±0.29**	9.86±0.09	12.76±1.25	**3.80±0.41**	**21.31±0.62**	**53.14±0.41**

	Scene 1 (4,333)			Scene 2 (3,079)			Scene 3 (2,612)		
Method	Pos. L2 (cm)	FDE (cm)	S@10 (%)	Pos. L2 (cm)	FDE (cm)	S@10 (%)	Pos. L2 (cm)	FDE (cm)	S@10 (%)
π0.5 [3]	11.63±0.25	18.58±0.41	25.49±0.93	14.79±0.21	25.43±0.38	22.59±0.40	17.95±0.17	30.08±0.19	10.40±0.28
SmolVLA [42]	7.35±0.24	13.13±0.40	48.17±1.31	10.22±0.42	19.47±0.98	36.05±3.57	11.69±0.07	19.94±0.15	20.41±0.57
OpenVLA-OFT [20]	**6.10±0.10**	10.98±0.22	53.55±1.89	8.80±0.10	17.48±0.18	38.75±0.20	10.83±0.19	20.01±0.45	24.50±1.21
GR00T N1.6 [35]	8.67±1.22	13.26±1.46	43.55±6.84	11.18±1.05	19.71±1.23	29.76±3.90	12.65±0.86	21.51±0.92	17.10±1.61
SPOT	6.71±0.26	**10.59±0.58**	**58.54±2.15**	**6.25±0.07**	**10.88±0.28**	**53.53±1.40**	**8.01±0.21**	**12.37±0.17**	**43.13±1.32**

SPOT leads overall translation and S@2/5/10 in Table 1. Relative to OpenVLA-OFT, its FDE margin grows from 0.39 cm in Scene 1 to 6.60/7.64 cm in Scenes 2/3; the S@10 margins are 4.99/14.78/18.63 percentage points. The increasing benefit with clutter suggests an instance-grounding bottleneck, not cross-scene generalization. OpenVLA-OFT retains better rotation and gripper accuracy, distinguishing *where* to move from fine pose control. Because task inputs and fine-tuning recipes differ, this comparison does not isolate architecture or establish how end-to-end VLA retraining would perform without instance identifiers.

5.3. Prompting Baselines

We compare five task specifications: NO PROMPT, which replaces object/target information with learned null tokens; BBOX PROMPT, which uses coordinate box tokens; VISUAL PROMPT, which renders the boxes on the first frame; TARGET ONLY, which provides only the target's visual prompt and bounding-box information; and BBOX+VISUAL, which combines coordinate and rendered prompts.

Grounding primarily reduces large endpoint errors (Table 2). BBox+Visual lowers No Prompt's Pos. L2/FDE by 2.49/6.61 cm and improves S@10 by 13.95 percentage points, with little S@2 change. Prompts thus resolve ambiguous associations more than refine already-close predictions. Visual Prompt leads several mean metrics, whereas BBox alone is weaker. Coordinate tokens improve stability: BBox+Visual has the highest S@10 and reduces its across-seed standard deviation from Visual Prompt's 3.17 to 0.41 percentage points. Target Only explains most endpoint gains; adding object context improves Pos. L2 and S@5/10, consistent with destination versus path specification. Appendix Table 13 reports significance tests.

5.4. Spatial Prompt Dependence and Robustness

To determine whether SPOT relies on spatial prompts rather than dataset priors, we perturb the spatial prompts at inference time while keeping the observation, motion history, ground truth, model checkpoint, and initial action noise fixed. For the shuffled and role-swapped conditions, we redraw the rendered visual prompt on the source-resolution first frame and update the corresponding coordinate tokens accordingly.

Table 3. Inference-time prompt interventions on the same seed BBox+Visual checkpoint. Observation, history, ground truth, and initial action noise are fixed. G-end denotes endpoint gripper-state accuracy after binarizing gripper width at 30mm. Combined@10 denotes the percentage of action chunks whose endpoint simultaneously satisfies FDE $\leq$ 10 cm, Rot. geo $\leq 20^\circ$, and correct gripper state. Both are offline endpoint metrics rather than closed-loop task-success rates.

Prompt condition	FDE (cm)	Rot. geo. (°)	G-end (%)	Combined@10 (%)
Shuffled object	13.8424	10.90	80.56	29.93
Shuffled target	18.4667	11.09	79.34	26.68
Object/target swap	22.0824	11.23	79.62	24.77
Correct BBox + visual	**11.4655**	**9.95**	**81.04**	**38.67**

Table 3 shows that shuffling the object/target raises FDE by 20.74%/61.04%, which indicates that target placement is substantially more challenging than object pickup. Swapping the object and target roles produces an even larger FDE increase of 92.60%. This degradation exceeds that caused by either shuffle, even though both regions remain visible, indicating that SPOT relies on their semantic roles rather than mere visual saliency. By contrast, Rot. geo. changes by at most 1.28°, and G-end changes by only 1.70%, suggesting that prompt corruption redirects the overall trajectory much more strongly than it affects local pose or gripper state. Additional robustness results under prompt scaling and jitter are provided in the Appendix.

5.5. Role-Specific Effects Across Execution

We segment each validation episode using ground-truth gripper width and EE translation speed into Approach (initial motion), Pre-grasp (final deceleration), Grasp/Occlusion (closing and initial lift), Transport (post-grasp transfer), and Place-

Table 2. Three-seed prompting results, reported as mean ± sample standard deviation. Lower is better for the four error metrics; higher is better for endpoint translation success. Paired-bootstrap confidence intervals are reported in Appendix Table 13.

Prompt	Pos. L2 (cm)	FDE (cm)	Rot. geo. (°)	Grip. L1 (mm)	S@2 (%)	S@5 (%)	S@10 (%)
No prompt	9.39±0.26	17.75±0.39	10.27±0.16	11.91±0.99	4.31±0.63	19.47±1.17	39.19±0.97
Target only	7.07±0.02	11.57±0.12	10.22±0.27	13.09±0.92	3.35±0.08	20.16±0.58	50.14±0.56
Visual	6.83±0.18	11.25±0.50	9.49±0.10	12.08±1.10	3.99±0.34	22.66±0.94	52.93±3.17
BBox	7.15±0.07	11.85±0.14	10.33±0.14	13.05±1.21	3.00±0.17	19.01±0.19	49.02±0.36
BBox + visual	6.90±0.10	11.14±0.29	9.86±0.09	12.76±1.25	3.80±0.41	21.31±0.62	53.14±0.41

Table 5. Pick/place endpoint analysis under two phase definitions. FDE is in centimeters; S@2/5/10 are offline endpoint criteria reported in percent.

Split	Condition	Pick FDE	Place FDE	Pick S@2/5/10	Place S@2/5/10
Task half	No prompt	10.76	27.32	**5.5/26.6**/56.0	2.5/7.7/14.7
	Target only	10.81	**12.40**	2.3/20.3/52.7	**3.0/14.8/43.8**
	BBox + visual	**9.59**	13.86	3.5/23.6/**61.6**	2.3/13.5/37.4
Contact ±15	No prompt	8.32	24.79	**9.4/39.1**/69.5	4.0/12.9/22.9
	Target only	9.78	**11.50**	3.4/26.2/58.7	**3.9/20.1/49.8**
	BBox + visual	**7.88**	13.46	5.7/32.6/**74.3**	2.7/18.2/40.9

ment (final target approach and release). Each trajectory chunk's FDE is assigned to the phase containing its future endpoint timestep. The detection thresholds, fallback rules, and segmentation diagnostics are provided in Appendix C.7.

Table 4. Five-stage endpoint analysis. Each chunk is grouped by its endpoint phase; phase definitions and segmentation details are given in Appendix C.7. Values are FDE (cm); lower is better.

Condition	Approach	Pre-grasp	Grasp/occl.	Transport	Placement
No prompt	12.41	9.42	7.05	15.63	25.41
Shuffled object	18.64	14.30	10.25	13.13	15.42
Shuffled target	**11.54**	**9.28**	7.55	16.69	27.21
Role swap	12.00	13.74	11.08	19.18	32.09
Target only	14.28	11.54	8.84	12.33	**11.36**
BBox + visual	12.16	9.79	**6.97**	**12.29**	12.83

BBox+Visual resembles No Prompt early but lowers Transport/Placement FDE by 21.4%/49.5% (Table 4). Observations and history constrain early motion, while grounding matters most for destination selection. Object shuffling primarily harms the first three phases; target shuffling raises Transport/Placement FDE by 35.8%/112.1%. Target Only leads Placement, suggesting that stage-aware fusion could downweight object cues after grasping. Object and target cues thus support acquisition and delivery differently rather than contributing uniformly.

We further aggregate performance into Pick and Place stages using two complementary definitions: a task-half split and contact-centered windows that include chunk endpoints within ±15 raw video frames (approximately ±0.5 s) of the detected pick or place contact. Stage boundaries are jointly estimated from the EE position and gripper width.

Both definitions agree (Table 5): BBox+Visual leads Pick FDE/S@10, but No Prompt retains higher Pick S@2/5. Prompts reduce gross pickup errors without improving the finest accuracy. During Place, BBox+Visual nearly halves No Prompt's FDE, while Target Only performs best. Agreement with the high-confidence-boundary subset supports role separation beyond a single segmentation rule, but does not establish closed-loop success.

Table 6. Autoregressive rollout results. We report episode-end FDE (cm, mean ± std over three seeds across 257 episodes) under replayed RGB. `GT` and `Pred` denote recorded and predicted states.

History / pose anchor	SPOT	No Prompt
`GT` / `GT`	4.55 ± 0.14	5.91 ± 0.29
`Pred` / `GT`	6.53 ± 0.24	9.36 ± 0.75
`Pred` / `Pred`	22.59 ± 0.78	41.30 ± 0.39

5.6. Autoregressive Rollout and Drift Analysis

To evaluate error accumulation, we predict consecutive, non-overlapping $H = 16$ chunks after a $K = 4$ recorded warm start. Table 6 compares recorded versus predicted history and current EE poses used to place each chunk in world coordinates. Here, FDE scores only the last valid point of each episode, even when the final chunk has fewer than 16 valid steps, and gives each episode equal weight. In contrast, Table 2 averages errors at the 16th predicted point over all complete sliding windows. The two FDE values therefore should not be compared directly; feedback effects are measured against the GT/GT control within Table 6. Appendix C.3 gives the scoring details and an example.

Propagating predicted pose anchors causes much larger drift than feeding back motion history alone (raising Episode-end FDE from 6.53 to 22.59 cm), indicating that compounding coordinate shifts dominate rollout errors. Spatial prompts substantially buffer against this drift, reducing cumulative Episode-end FDE by 45.3% and waypoint error by 27.6% over No Prompt. Moreover, shuffling the target under feedback errors causes a sharp collapse in placement accuracy, highlighting that target cues serve as a late-stage corrective anchor when state estimates drift. Nevertheless, first-frame prompts cannot fully eliminate open-loop drift, especially

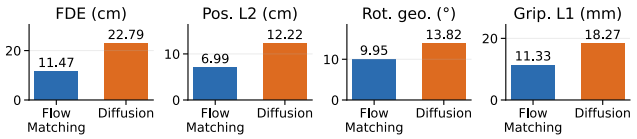

Figure 4. **Trajectory head ablation.** Flow matching consistently outperforms diffusion. Lower is better.

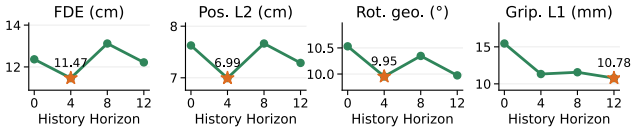

Figure 5. **History horizon ablation.** Stars mark the best value for each metric. Lower is better.

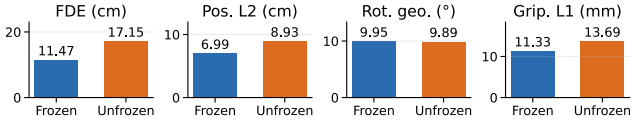

Figure 6. **Visual backbone tuning ablation.** The frozen DINOv2 encoder outperforms end-to-end tuning. Lower is better.

since replayed RGB observations do not update with predicted poses to allow visual closed-loop recovery.

5.7. Architecture Ablations

Trajectory head. With the same decoder architecture, flow matching reduces diffusion's FDE/Pos. L2 by 49.7%/42.8% and improves rotation and gripper errors (Figure 4). Gains across endpoint and waypoint metrics favor the tested generative formulation, not added decoder capacity.

History horizon. In Figure 5, $K = 4$ lowers FDE/Grip. L1 by 7.3%/26.7% relative to $K = 0$. Increasing K to 8 worsens all metrics; $K = 12$ leads Grip. L1 but remains worse than $K = 4$ in translation. The larger gripper gain suggests that recent motion helps infer execution phase; longer context offers no consistent gain.

Backbone tuning. As shown in Figure 6 Freezing DINOv2 improves FDE/Pos. L2/Grip. L1, whereas fine-tuning changes rotation by less than $0.1°$. This favors frozen features under our training recipe; validation errors alone cannot distinguish overfitting from optimization sensitivity.

These results motivate our default configuration. Further interpretation and encoder tradeoffs appear in Appendix F.

5.8. Qualitative Evaluation

Figure 7 shows that stitched predictions capture the overall path shape, though mid-trajectory errors (>10 cm) exceed the final error (5.25 cm), showing the need for both path (Pos. L2) and endpoint (FDE) metrics. In a single chunk (Figure 8), position error grows over the 16 steps even though rotation and gripper errors stay low, indicating that pose and gripper

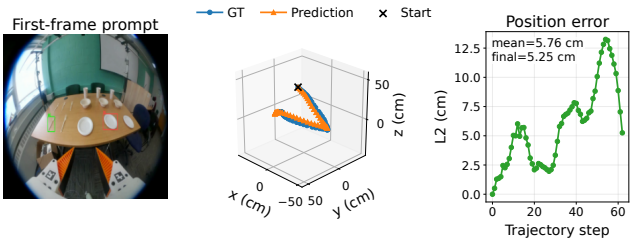

Figure 7. **Full trajectory visualization.** First-frame prompt (left), stitched predictions versus ground truth (middle), and per-frame position errors (right).

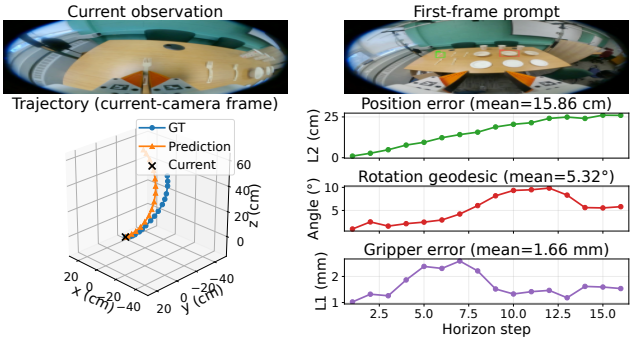

Figure 8. **First chunk visualization.** Current RGB (top left), first-frame prompt (top right), predicted/GT chunks (bottom left), and position, rotation-geodesic, and gripper errors (bottom right).

control are decoupled from 3D spatial reach. The paired images also show complementary inputs: the first-frame prompt defines the global task goals, while the current view guides local execution. Note that these plots use ground-truth anchors rather than autoregressive rollouts (Table 6); see the Appendix for more examples.

6. Conclusion

We introduce *Spatially Prompted Visual Trajectory Prediction* (SP-VTP), EgoSPT with spatial grounding and recovered 3D motion, and SPOT, which fuses visual and coordinate prompts with observation and motion history. Across three seeds, spatial prompts reduce translation and endpoint errors, with Visual and BBox+Visual prompts offering complementary advantages. Controlled interventions further localize object-cue effects to approach/grasp and target-cue effects to transport/placement. These findings apply to fork pick-and-place in the three evaluated scenes; transfer to other objects and tasks and closed-loop execution remain untested.

Acknowledgement

We thank the undergraduate students at MSU who helped with data collection: Grace Chacko, Aaron Zheng, Dong Duong, Siddhesh Kulkarni, Aneeshwa Chittanuri, Arman Khan, and Atchyutram Chundru.

References

[1] Prithviraj Banerjee, Sindi Shkodrani, Pierre Moulon, Shreyas Hampali, Shangchen Han, Fan Zhang, Linguang Zhang, Jade Fountain, Edward Miller, Selen Basol, et al. Hot3d: Hand and object tracking in 3d from egocentric multi-view videos. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 7061–7071. IEEE, 2025. 2

[2] Wentao Bao, Lele Chen, Libing Zeng, Zhong Li, Yi Xu, Junsong Yuan, and Yu Kong. Uncertainty-aware state space transformer for egocentric 3d hand trajectory forecasting. In *Int. Conf. Comput. Vis.*, pages 13702–13711, 2023. 2

[3] Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, brian ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, pages 17–40. PMLR, 2025. 2, 5, 6

[4] Cheng Chi, Zhenjia Xu, Chuer Pan, Eric Cousineau, Benjamin Burchfiel, Siyuan Feng, Russ Tedrake, and Shuran Song. Universal manipulation interface: In-the-wild robot teaching without in-the-wild robots. In *Robotics: Science and Systems*, 2024. 2, 3

[5] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *Int. J. Robot. Res.*, 44(10-11):1684–1704, 2025. 2, 5

[6] Dima Damen, Hazel Doughty, Giovanni Maria Farinella, Antonino Furnari, Evangelos Kazakos, Jian Ma, Davide Moltisanti, Jonathan Munro, Toby Perrett, Will Price, et al. Rescaling egocentric vision: Collection, pipeline and challenges for epic-kitchens-100. *Int. J. Comput. Vis.*, 130(1):33–55, 2022. 2

[7] Irving Fang, Yuzhong Chen, Yifan Wang, Jianghan Zhang, Qiushi Zhang, Jiali Xu, Xibo He, Weibo Gao, Hao Su, Yiming Li, et al. Egopat3dv2: Predicting 3d action target from 2d egocentric vision for human-robot interaction. In *IEEE Int. Conf. Robot. Autom.*, pages 3036–3043. IEEE, 2024. 2

[8] Ankit Goyal, Valts Blukis, Jie Xu, Yijie Guo, Yu-Wei Chao, and Dieter Fox. Rvt2: Learning precise manipulation from few demonstrations. *Robotics: Science and Systems*, 2024. 2

[9] Kristen Grauman, Andrew Westbury, Eugene Byrne, Zachary Chavis, Antonino Furnari, Rohit Girdhar, Jackson Hamburger, Hao Jiang, Miao Liu, Xingyu Liu, et al. Ego4d: Around the world in 3,000 hours of egocentric video. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 18995–19012, 2022. 2

[10] Kristen Grauman, Andrew Westbury, Lorenzo Torresani, Kris Kitani, Jitendra Malik, Triantafyllos Afouras, Kumar Ashutosh, Vijay Baiyya, Siddhant Bansal, Bikram Boote, et al. Ego-exo4d: Understanding skilled human activity from first-and third-person perspectives. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 19383–19400, 2024. 2

[11] Jiayuan Gu, Sean Kirmani, Paul Wohlhart, Yao Lu, Montserrat Gonzalez Arenas, Kanishka Rao, Wenhao Yu, Chuyuan Fu, Keerthana Gopalakrishnan, Zhuo Xu, Priya Sundaresan, Peng Xu, Hao Su, Karol Hausman, Chelsea Finn, Quan Vuong, and Ted Xiao. Rt-trajectory: Robotic task generalization via hindsight trajectory sketches. In *International Conference on Learning Representations*, pages 44918–44941, 2024. 2

[12] Harsh Gupta, Xiaofeng Guo, Huy Ha, Chuer Pan, Muqing Cao, Dongjae Lee, Sebastian Scherer, Shuran Song, and Guanya Shi. Umi-on-air: Embodiment-aware guidance for embodiment-agnostic visuomotor policies, 2025. 2, 3

[13] Huy Ha, Yihuai Gao, Zipeng Fu, Jie Tan, and Shuran Song. UMI-on-legs: Making manipulation policies mobile with manipulation-centric whole-body controllers. In *Conf. Robot Learn.*, 2024. 2

[14] Masashi Hatano, Ryo Hachiuma, and Hideo Saito. Emag: Ego-motion aware and generalizable 2d hand forecasting from egocentric videos. In *Eur. Conf. Comput. Vis.*, pages 119–136, 2024. 2

[15] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Adv. Neural Inform. Process. Syst.*, pages 6840–6851, 2020. 5

[16] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. Voxposer: Composable 3d value maps for robotic manipulation with language models. In *Conf. Robot Learn.*, 2023. 2

[17] Yunfan Jiang, Agrim Gupta, Zichen Zhang, Guanzhi Wang, Yongqiang Dou, Yanjun Chen, Li Fei-Fei, Anima Anandkumar, Yuke Zhu, and Linxi Fan. VIMA: Robot manipulation with multimodal prompts. In *Proceedings of the 40th International Conference on Machine Learning*, pages 14975–15022. PMLR, 2023. 2

[18] Simar Kareer, Dhruv Patel, Ryan Punamiya, Pranay Mathur, Shuo Cheng, Chen Wang, Judy Hoffman, and Danfei Xu. Egomimic: Scaling imitation learning via egocentric video. In *IEEE Int. Conf. Robot. Autom.*, pages 13226–13233. IEEE, 2025. 2

[19] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. In *Conf. Robot Learn.*, 2024. 1, 2

[20] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025. 5, 6

[21] Taein Kwon, Bugra Tekin, Jan Stühmer, Federica Bogo, and Marc Pollefeys. H2o: Two hands manipulating objects for first person interaction recognition. In *Int. Conf. Comput. Vis.*, pages 10138–10148, 2021. 2

[22] Yiming Li, Ziang Cao, Andrew Liang, Benjamin Liang, Luoyao Chen, Hang Zhao, and Chen Feng. Egocentric prediction of action target in 3d. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 20971–20980. IEEE, 2022. 2

[23] Yi Li, Yuquan Deng, Jesse Zhang, Joel Jang, Marius Memmel, Raymond Yu, Caelan Reed Garrett, Fabio Ramos, Dieter Fox, Anqi Li, Abhishek Gupta, and Ankit Goyal. Hamster: Hierarchical action models for open-world robot manipulation. *arXiv preprint arXiv:2502.05485*, 2025. 1

[24] Fanqi Lin, Yingdong Hu, Pingyue Sheng, Chuan Wen, Jiacheng You, and Yang Gao. Data scaling laws in imitation learning for robotic manipulation. In *Int. Conf. Learn. Represent.*, 2025. 2

[25] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *Int. Conf. Learn. Represent.*, 2023. 5

[26] Kehui Liu, Zhongjie Jia, Yang Li, Pengan Chen, Song Liu, Xin Liu, Pingrui Zhang, Haoming Song, Xinyi Ye, Nieqing Cao, et al. Fastumi-100k: Advancing data-driven robotic manipulation with a large-scale umi-style dataset. *arXiv preprint arXiv:2510.08022*, 2025. 2

[27] Shaowei Liu, Subarna Tripathi, Somdeb Majumdar, and Xiaolong Wang. Joint hand motion and interaction hotspots prediction from egocentric videos. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 3282–3292, 2022. 2

[28] Zeyi Liu, Cheng Chi, Eric Cousineau, Naveen Kuppuswamy, Benjamin Burchfiel, and Shuran Song. Maniwav: Learning robot manipulation from in-the-wild audio-visual data. In *Conf. Robot Learn.*, pages 947–962. PMLR, 2025. 2

[29] Yunhao Luo and Yilun Du. Grounding video models to actions through goal conditioned exploration. In *Int. Conf. Learn. Represent.*, 2025. 2

[30] Corey Lynch and Pierre Sermanet. Language conditioned imitation learning over unstructured data. In *Robotics: Science and Systems*, 2021. 1, 2

[31] Junyi Ma, Xieyuanli Chen, Jingyi Xu, and Hesheng Wang. Diff-ip2d: Diffusion-based hand-object interaction prediction on egocentric videos. In *IEEE/RSJ Int. Conf. Intell. Robots Syst.*, pages 4291–4298. IEEE, 2025. 2

[32] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021. 4

[33] Ruiqian Nai, Boyuan Zheng, Junming Zhao, Haodong Zhu, Sicong Dai, Zunhao Chen, Yihang Hu, Yingdong Hu, Tong Zhang, Chuan Wen, et al. Humanoid manipulation interface: Humanoid whole-body manipulation from robot-free demonstrations. *arXiv preprint arXiv:2602.06643*, 2026. 2

[34] Ashvin V Nair, Vitchyr Pong, Murtaza Dalal, Shikhar Bahl, Steven Lin, and Sergey Levine. Visual reinforcement learning with imagined goals. *Adv. Neural Inform. Process. Syst.*, 31, 2018. 2

[35] NVIDIA GEAR Team. GR00T N1.6: An improved open foundation model for generalist humanoid robots. NVIDIA Research, 2025. Released December 15, 2025. 5, 6

[36] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Charles Xu, Jianlan Luo, Tobias Kreiman, You Liang Tan, Lawrence Yunliang Chen, Pannag Sanketi, Quan Vuong, Ted Xiao, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems*, Delft, Netherlands, 2024. 2

[37] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. Dinov2: Learning robust visual features without supervision. *Trans. Mach. Learn. Res.*, 2024. 4

[38] Moritz Reuss, Maximilian Li, Xiaogang Jia, and Rudolf Lioutikov. Goal-conditioned imitation learning using score-based diffusion policies. In *Robotics: Science and Systems*, 2023. 2

[39] Mingyo Seo, H Andy Park, Shenli Yuan, Yuke Zhu, and Luis Sentis. Legato: Cross-embodiment imitation using a grasping tool. *IEEE Robot. Autom. Lett.*, 10(3):2854–2861, 2025. 2

[40] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Cliport: What and where pathways for robotic manipulation. In *Conf. Robot Learn.*, pages 894–906. PMLR, 2022. 1, 2

[41] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Perceiver-actor: A multi-task transformer for robotic manipulation. In *Conf. Robot Learn.*, pages 785–799. PMLR, 2023. 1, 2

[42] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. Smolvla: A vision-language-action model for affordable and efficient robotics, 2025. 5, 6

[43] Aravind Srinivas, Allan Jabri, Pieter Abbeel, Sergey Levine, and Chelsea Finn. Universal planning networks: Learning generalizable representations for visuomotor control. In *Int. Conf. Machi. Learn.*, pages 4732–4741. PMLR, 2018. 2

[44] Priya Sundaresan, Suneel Belkhale, Dorsa Sadigh, and Jeannette Bohg. Kite: Keypoint-conditioned policies for semantic manipulation. In *Proceedings of The 7th Conference on Robot Learning*, pages 1006–1021. PMLR, 2023. 2

[45] Matthew Tancik, Pratul Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *Adv. Neural Inform. Process. Syst.*, pages 7537–7547, 2020. 4

[46] Hao Tang, Kevin J Liang, Kristen Grauman, Matt Feiszli, and Weiyao Wang. Egotracks: A long-term egocentric visual object tracking dataset. *Adv. Neural Inform. Process. Syst.*, 36:75716–75739, 2023. 2

[47] Tony Tao, Mohan Kumar Srirama, Jason Jingzhou Liu, Kenneth Shaw, and Deepak Pathak. Dexwild: Dexterous human interactions for in-the-wild robot policies. *Robotics: Science and Systems*, 2025. 2

[48] Gemini Robotics Team, Abbas Abdolmaleki, Saminda Abeyruwan, Joshua Ainslie, Jean-Baptiste Alayrac, Montserrat Gonzalez Arenas, Ashwin Balakrishna, Nathan Batchelor, Alex Bewley, Jeff Bingham, et al. Gemini robotics 1.5: Pushing the frontier of generalist robots with advanced embod-

ied reasoning, thinking, and motion transfer. *arXiv preprint arXiv:2510.03342*, 2025. 1

[49] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Adv. Neural Inform. Process. Syst.*, 2017. 4

[50] Zixuan Wang, Yuxin Chen, Yuqi Liu, Jinhui Ye, Pengguang Chen, Changsheng Lu, Shu Liu, and Jiaya Jia. Vp-vla: Visual prompting as an interface for vision-language-action models. *arXiv preprint arXiv:2603.22003*, 2026. 1, 2

[51] Yifei Wei, Linqing Zhong, Yi Liu, Yuxiang Lu, Xindong He, Maoqing Yao, and Guanghui Ren. Libra-vla: Achieving learning equilibrium via asynchronous coarse-to-fine dual-system. *arXiv preprint arXiv:2604.24921*, 2026.

[52] Zhenyu Wu, Yuheng Zhou, Xiuwei Xu, Ziwei Wang, and Haibin Yan. Momanipvla: Transferring vision-language-action models for general mobile manipulation. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 1714–1723, 2025. 1

[53] Mengda Xu, Han Zhang, Yifan Hou, Zhenjia Xu, Linxi Fan, Manuela Veloso, and Shuran Song. Dexumi: Using human hand as the universal manipulation interface for dexterous manipulation. In *Conf. Robot Learn.*, pages 437–459. PMLR, 2025. 2

[54] Jianwei Yang, Reuben Tan, Qianhui Wu, Ruijie Zheng, Baolin Peng, Yongyuan Liang, Yu Gu, Mu Cai, Seonghyeon Ye, Joel Jang, Yuquan Deng, and Jianfeng Gao. Magma: A foundation model for multimodal ai agents. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 14203–14214, 2025. 1

[55] Ruihan Yang, Huazhe Xu, YI WU, and Xiaolong Wang. Multi-task reinforcement learning with soft modularization. In *Adv. Neural Inform. Process. Syst.*, pages 4767–4777. Curran Associates, Inc., 2020. 1, 2

[56] Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Conf. Robot Learn.*, pages 1094–1100. PMLR, 2020. 1, 2

[57] Michał Zawalski, William Chen, Karl Pertsch, Oier Mees, Chelsea Finn, and Sergey Levine. Robotic control via embodied chain-of-thought reasoning. In *Conf. Robot Learn.*, pages 3157–3181. PMLR, 2025. 1

[58] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems*, 2023. 2

[59] Zhaxizhuoma, Kehui Liu, Chuyue Guan, Zhongjie Jia, Ziniu Wu, Xin Liu, Tianyu Wang, Shuai Liang, Pengan Chen, Pingrui Zhang, Haoming Song, Delin Qu, Dong Wang, Zhigang Wang, Nieqing Cao, Yan Ding, Bin Zhao, and Xuelong Li. Fastumi: A scalable and hardware-independent universal manipulation interface with dataset. *arXiv*, 2025. 2

[60] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R Sanketi, Grecia Salazar, Michael S Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J Joshi, Alex Irpan, brian ichter, Jasmine Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Kumar Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Marcin Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conf. Robot Learn.*, 2023. 1, 2

Spatially Prompted Visual Trajectory Prediction for Egocentric Manipulation

Supplementary Material

Figure 9. More visualization demonstrations of modified UMI device.

A. Dataset Documentation

Collection setup. EgoSPT contains 2,841 egocentric pick-and-place episodes collected with a modified UMI (see Fig. 9) by nine trained experts. Each episode records an RGB video and tracking streams. The recording interface stores raw episode videos as `camera_<i>.mp4` and per-camera arrays such as pose, receive time, capture time, and intrinsics. In the processed dataset used by SPOT, `camera_1.mp4` is the RGB video consumed by the vision model, while the tracking stream is used to recover time-aligned 6-DoF EE motion. Videos are recorded at 1920×1080 and 30 fps, and are temporally subsampled with stride 3 for trajectory prediction.

Trajectory processing pipeline. We turn each recording into a time-aligned RGB video, EE trajectory, gripper-width sequence, and first-frame prompts. First, we synchronize the video and tracking timestamps and interpolate the EE poses at valid video-frame times. We then detect the gripper's ArUco tags, calibrate their separation as gripper width, and interpolate missing width measurements. Finally, we save the processed recording and attach the object and target boxes. Each processed episode has the layout

```
<data_root>/<scene>/<task>/recording_
    output_processed/<episode>/
```

containing `camera_1.mp4`, `valid_indices`, `pose_interp`, and `gripper_widths`. The raw videos and processed trajectories are organized under `umi_day/output_data`, and first-frame prompt annotations are stored separately in `annotations_merged.json`.

Time alignment and trajectory recovery. We first correct the video and tracking timestamps for their configured delays. Only RGB frames within the shared recording period are retained; `valid_indices` identifies them. We use linear interpolation for position and spherical linear interpolation for rotation. A calibrated camera-to-EE transform converts camera poses to EE poses. Gripper width comes from the finger tags, using a separate gripper-range recording for calibration. Interpolating these measurements at the retained frame times produces `pose_interp` and `gripper_widths` aligned to the RGB video.

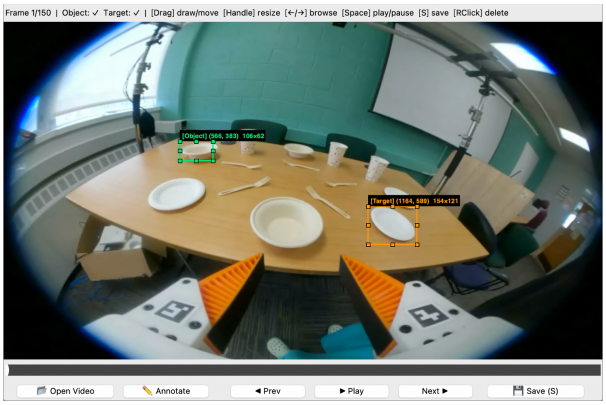

Figure 10. **Annotation interface.** Annotators label the manipulated object and target region on the first egocentric frame. The resulting bounding boxes are stored in the annotation JSON and used as spatial prompts for SP-VTP.

Annotations and sample construction. Nine trained experts are assigned the collected videos and annotate the object and target bounding boxes using our annotation tool, as shown in Fig. 10. The tool is a lightweight video bounding-box annotator designed for frame-0 task specification: annotators open an episode video, jump to the first frame, and draw exactly two boxes, where the first box denotes the manipulated object and the second denotes the target region. It also supports video browsing, playback, zooming, box editing, and in-place correction of merged annotations. Annotations are saved in JSON format with both the ordered box list and explicit `object`/`target` fields in original video pixel coordinates. The annotation results are merged and stored in `annotations_merged.json`.

After annotation, we manually check each bounding box with a separate annotation modification tool shown in Fig. 11. This tool loads videos from `annotations_merged.json`, supports direct navigation through annotated videos, allows frame browsing and playback, and restricts box editing to frame 0. Annotators can move, resize, create, or delete boxes, and the corrected object/target boxes are written back to the original merged annotation file in place.

The annotation file links each video to its object and target boxes in the first frame's original pixel coordinates. The loader uses the video's scene, task, and episode name

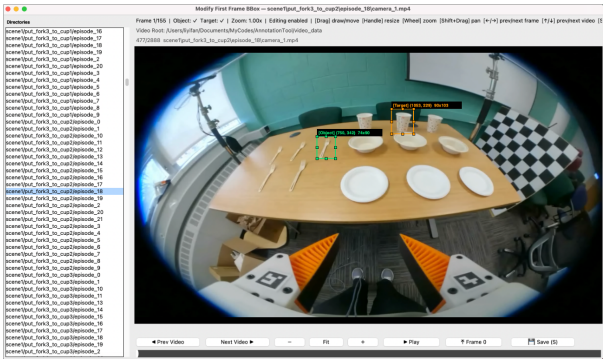

Figure 11. **Annotation modification interface.** After initial annotation, annotators inspect each video and correct frame-0 object/target bounding boxes directly in the merged annotation file.

to find the processed recording. It skips recordings with missing videos, prompts, or zarr arrays, or too few valid frames. At timestep t, one sample contains the first image, current image, normalized prompts, past actions, and future actions. Future poses are expressed relative to the current EE pose:

$$T_{\text{rel}}(t,h) = T_t^{-1} T_{t+h},$$

The resulting action contains relative translation, 6D rotation, and absolute gripper width. Past poses are also expressed relative to the current pose. The returned fields are `first_frame`, `current_frame`, `prompt_object`, `prompt_target`, `action_history`, `future_actions`, and `is_final_chunk`.

Preprocessing and scale. The default preprocessing uses image size 224×224, bounding-box prompts, action horizon $H = 16$, history horizon $K = 4$, and stride 3. This produces 112,856 sliding-window trajectory samples in total, including 102,832 training samples from 2,584 episodes and 10,024 validation samples from 257 episodes. The mean episode length after preprocessing is 59.7 trajectory steps, and each episode yields 39.7 trajectory samples on average. Across all samples, the mean relative-translation magnitude is 21.59 cm, the mean final displacement is 35.29 cm, and the mean gripper width is 30.3 mm.

Scene and task coverage. The data cover five visually similar forks and nine target receptacles across three scenes. Scene 1 uses structured layouts, Scene 2 uses a cluttered layout, and Scene 3 uses diverse cluttered subscenes with fewer demonstrations per configuration. The full dataset contains 931 episodes / 43,972 samples from Scene 1, 919 episodes / 35,902 samples from Scene 2, and 991 episodes / 32,982 samples from Scene 3. Scene 1 and Scene 2 are organized around object–target tasks such as `put_fork1_to_plate1`; Scene 3 is organized into 22 cluttered subscenes, each covering the object–target combinations with sparse demonstrations. The train/validation split is performed at the episode/task level while preserving scene coverage; the scene-distribution total variation between splits is 0.0015. All reported experiments therefore evaluate both overall validation performance and per-scene behavior.

Quality checks and statistics files. Before training, we verify that every annotation key maps to an existing processed episode, each processed episode has `camera_1.mp4`, `valid_indices`, `pose_interp`, and `gripper_widths`, each episode has enough valid frames for $K + H + 1$, the pose and gripper arrays share compatible temporal indexing, and first-frame boxes are valid in the original image coordinate system. We also check that scene/task/episode names in the annotations match directory names after path normalization and that the gripper calibration and tag detections are available for width interpolation.

We generate auxiliary statistics files to support reproducibility and data inspection. `summary.json` stores the machine-readable dataset summary, `episode_stats.csv` stores per-episode lengths, sample counts, and video metadata, `count_stats.csv` stores scene/task/scene-task counts, `prompt_stats.csv` stores bounding-box size and location statistics, `image_stats.csv` stores sampled RGB statistics, and `outliers_*.csv` stores samples with the largest final-displacement values for train and validation splits. These files are used to verify dataset scale, split quality, prompt validity, and trajectory target ranges. These files are provided in the supplementary material.

B. Model Architecture Details

Policy interface. The implementation is centered on `VisionTrajPolicy`, which maps a first-frame task specification and a current execution state to a future action chunk:

$$(\text{first frame}, \text{prompt}, \text{current frame}, \text{history}) \rightarrow A_t \in \mathbb{R}^{H \times 10}.$$

We use $H = 16$ by default. Each action vector is $[\Delta x, \Delta y, \Delta z, \text{rot6d}, g]$, where the pose component is expressed relative to the current camera/EE frame and g is the gripper width. Architecturally, the policy has three parts: the task encoder builds prompt-conditioned first-frame tokens, the observation encoder builds current-state tokens, and the action head decodes future trajectory tokens from their concatenation.

Prompt forms. The dataset loader exposes seven task-input variants. `bbox` uses a raw first frame with object/target bounding-box coordinates; `point` replaces each box with its center; `none` removes coordinate prompts and uses learned null prompt tokens; `vision_bbox` renders object/target boxes on the first frame without coordinate

prompts; and `vision_bbox_and_bbox` combines rendered boxes with coordinate tokens. `object_only` and `target_only` retain only the corresponding rendered box and coordinate prompt. The final SPOT configuration uses `vision_bbox_and_bbox`, so the task is represented in both image space and coordinate-token form.

Shared visual backbone. The task and observation encoders share the visual backbone. It is implemented with `timm` and returns N image tokens of width C for each of the B batch samples. The default is `vit_base_patch14_dinov2`, with $C = 768$ and policy dimension $D = 768$. The same interface supports SAM-Base, Perception Encoder-Base, SigLIP-Base, and EVA2-Base. A frozen backbone stays in evaluation mode and is excluded from the optimizer; an unfrozen backbone is trained with the rest of the policy.

Task encoder. The task encoder first applies the shared visual backbone to the first frame, optionally after visual prompt rendering. Image tokens are projected to dimension D and receive image type embeddings. Coordinate prompts are encoded separately: a bounding box contributes two corner tokens per retained role, a point prompt contributes one token per retained role, and the no-prompt setting uses two learned null tokens. Prompt tokens receive role/type embeddings before fusion.

The task encoder combines prompts with the first image using a Transformer decoder with 4 layers and 12 attention heads. The prompt tokens query the image tokens to extract information about the selected regions. The output contains one image-summary token followed by the updated prompt tokens. With the default object and target boxes, this gives five task tokens: one image summary and four box-corner tokens.

Observation encoder and conditioning sequence. The observation encoder applies the shared image encoder and projection to the current frame. If $K > 0$, a small MLP, `Linear(10,D)-GELU-Linear(D,D)`, maps each past 10D action to a token. Learned embeddings indicate its position in the history and distinguish it from image tokens. The observation sequence contains the current image tokens followed by these history tokens.

The final conditioning sequence is

$$Z_{\text{cond}} = [Z_{\text{task}}; Z_{\text{obs}}],$$

so the trajectory head can attend to the task specified in the first frame, the current image, and recent motion when predicting the next action chunk.

C. Training and Evaluation Details

C.1. Training Details

Default configuration. The SPOT prompting baselines and architecture ablations change one factor at a time from the default configuration. The default model uses `vision_bbox_and_bbox` prompts, a frozen DINOv2 ViT-B/14 visual encoder, policy dimension $D = 768$, cross-attention task fusion with 4 layers and 12 heads, history horizon $K = 4$, a 6-layer 12-head trajectory decoder, and a flow-matching head. The three-seed prompting runs use learning rate 2×10^{-4}, effective global batch size 128, 30 epochs, validation every 2 epochs, 4 dataloader workers, and bf16 mixed precision. To improve data-loading efficiency while preserving the global batch size, seed 42 uses 8 GPUs with 16 samples per GPU, seed 0 uses 2 GPUs with 64 samples per GPU, and seed 1 uses one GPU with batch size 128. Predecoded frame caching changes only I/O and was verified to return tensors identical to direct video decoding.

Split construction. We group episodes by scene and task. Within each scene, a seeded random shuffle selects the validation tasks according to the validation ratio. Every episode of a selected task goes to validation, so windows from that task are not divided between training and validation. The main training process saves the episode keys, scene/task names, and paths in `split_manifest.json`. We check that no episode appears in both splits.

Optimization. We use `accelerate` for distributed training and AdamW to update only trainable parameters. The frozen visual backbone is therefore not updated. After synchronizing gradients, we clip their norm to 1.0. Learning-rate schedules use `diffusers.optimization.get_scheduler`; the implementation supports constant, warmup, linear, cosine, cosine-with-restarts, and polynomial schedules.

Action normalization. Before optimizer construction, the policy computes per-action-dimension mean and standard deviation from training samples. Standard deviations are clamped to at least 10^{-4}. During training, future action chunks are normalized as $(a - \mu)/\sigma$ before being passed to the trajectory head; predictions are denormalized before evaluation or visualization.

Flow-matching head. The default flow-matching head predicts velocity over normalized action chunks. During training, time t is sampled from a Beta distribution with $\alpha = 1.5$ and $\beta = 1.0$, then clamped away from zero by $t = 0.999t + 0.001$. Given a clean action chunk x_0 and Gaussian noise ϵ, the interpolated sample is $x_t = (1 - t)x_0 + t\epsilon$, and the target velocity is $\epsilon - x_0$. Time is encoded with a continuous sinusoidal embedding over periods from 0.004 to 4.0, followed by a two-layer MLP.

Diffusion head. The alternative diffusion head uses a DDPM scheduler with the `squaredcos_cap_v2` beta schedule during training. It predicts either noise or the clean sample depending on the configured prediction type. The diffusion and flow-matching heads share the same high-level decoder structure: trajectory tokens with a time embedding attend to the conditioning tokens through a Transformer

decoder and output an action chunk.

C.2. Evaluation Details

Validation protocol. During training, we check a limited number of validation batches every two epochs. The final evaluation uses all validation samples and reports both overall and per-scene results. In this standard evaluation, each window receives the recorded current RGB frame and the previous $K = 4$ recorded actions. It predicts the next $H = 16$ actions independently: its predictions are not passed to another window. Future ground-truth (GT) actions are used only for scoring. Section C.3 describes the separate experiment that feeds predictions back into later queries.

Inference. For the default flow-matching head, inference starts from Gaussian noise at $t = 1$ and integrates to $t = 0$ using forward Euler with 10 steps. The final evaluation uses deterministic initial action noise with sample seed 0. For the diffusion head, inference uses a DDIM scheduler with the same `squaredcos_cap_v2` beta schedule used during training.

Position and gripper errors. Let N be the number of evaluated windows and $H = 16$ the number of future waypoints in each window. A hat denotes a prediction; the corresponding symbol without a hat denotes its recorded target. Position $p \in \mathbb{R}^3$ and gripper width g are stored in meters. For window i and future step h, define

$$e^p_{i,h} = 100\|\hat{p}_{i,h} - p_{i,h}\|_2, \qquad e^g_{i,h} = 1000|\hat{g}_{i,h} - g_{i,h}|.$$

Thus e^p is in centimeters and e^g is in millimeters. Pos. L2 averages position error over every waypoint, whereas Chunk-FDE@16 uses only the 16th waypoint of each window:

$$\text{Pos. L2} = \frac{1}{NH}\sum_{i,h} e^p_{i,h},$$

$$\text{Chunk-FDE@16} = \frac{1}{N}\sum_{i} e^p_{i,16}.$$

Grip. L1 averages $e^g_{i,h}$ over the same NH waypoints. Table 2 and the other fixed-horizon tables label this metric simply FDE. Here we call it Chunk-FDE@16 to make clear that it scores a chunk endpoint, not the end of an entire episode. Section C.3 explains the episode-end FDE used for rollout.

Rotation error. Each action represents orientation by a 6D vector r. We split it into two 3D vectors (a, b) and turn them into a rotation matrix using Gram–Schmidt: normalize a to obtain u, remove the component of b parallel to u and normalize the remainder to obtain v, then set $R = [u, v, u \times v]$. Given predicted and recorded rotations $\hat{R}_{i,h}$ and $R_{i,h}$, the angle between them is

$$e^R_{i,h} = \frac{180}{\pi}\cos^{-1}\left(\text{clip}\left(\frac{\text{tr}(\hat{R}^\top_{i,h}R_{i,h}) - 1}{2}, -1, 1\right)\right).$$

Rot. geo. is this angle in degrees, averaged over all NH waypoints. Endpoint Rot. geo. uses only step 16. R@τ is the percentage of windows whose step-16 rotation error is at most $\tau°$, for $\tau \in \{10, 20\}$. Some diagnostic tables also report Rot6D L2, the average $\|\hat{r}_{i,h} - r_{i,h}\|_2$. This is a dimensionless error between the 6D vectors, not an angle, and is not used to compute Combined success.

Endpoint criteria: S@X, G-end, and Combined@X. These metrics all score step 16 in the standard window evaluation. S@X is the percentage of windows whose position error is at most X cm:

$$\text{S@}X = \frac{100}{N}\sum_i \mathbb{1}[e^p_{i,16} \le X], \quad X \in \{2, 5, 10\}.$$

G-end checks whether the predicted and recorded gripper states agree. A width above 30 mm is called open; all other widths are called closed. G-end is the percentage of matching step-16 states. Combined@X is stricter: the same endpoint must pass the position, rotation, and gripper checks together:

$$e^p_{i,H} \le X, \qquad e^R_{i,H} \le 20°,$$
$$\mathbb{1}[1000\hat{g}_{i,H} > 30] = \mathbb{1}[1000g_{i,H} > 30].$$

Here $X \in \{5, 10\}$ cm, the rotation limit is $20°$, and multiplying g by 1000 converts meters to millimeters. These percentages measure prediction accuracy, not whether a robot actually completed the task.

Averaging and saved results. The standard evaluation averages over all 10,024 validation windows. When a phase analysis instead gives each episode equal weight, its caption or protocol says so. For three-seed results, we evaluate the same samples for each seed, then report the mean and sample standard deviation across seeds. Overall and per-scene outputs are saved as `summary.json` and `metrics.csv` under `eval_metrics`. The full-episode figures stitch predictions from GT-anchored windows; they are not the cumulative rollout experiment below.

C.3. Sequential Rollout with Recorded RGB

What is reused and what changes? We reuse the BBox+Visual and independently trained No Prompt checkpoints for seeds 42/0/1; no model is retrained. Shuffled Object and Shuffled Target use the corresponding BBox+Visual checkpoint, changing both the drawn box and its coordinate tokens. All four conditions use the same 257 validation episodes. With three seeds and three feedback modes, this gives 36 evaluations.

When does the model predict? Each episode starts with the recorded history through step $K = 4$. The model then predicts at steps $4, 20, 36, \ldots$, advancing by a full $H = 16$ chunk each time. The first-frame prompt stays fixed. If fewer than 16 future steps remain, we keep only the predictions for those valid steps. For example, a query at step 20 predicts

steps 21–36; if the episode ends at step 29, only predictions 21–29 are scored. Each evaluation has 986 queries: 746 complete chunks and 240 short final chunks. Together they cover 13,906 future waypoints, each scored once.

What do GT and Pred mean? The two entries in Table 6 specify the source of the past history and the current pose anchor, respectively. The pose anchor is simply the EE position and orientation used to place a relative prediction in world coordinates.

- **GT/GT:** use the recorded history and current pose at every query. This is the reference condition without prediction feedback.
- **Pred/GT:** use earlier predictions as history, but use the recorded current pose to place every new chunk. This tests history errors while correcting the pose anchor each time.
- **Pred/Pred:** use predicted history and the previous chunk's predicted final pose as the next anchor. After initialization, there is no reset to a recorded pose, so position and orientation errors can carry over between chunks.

Each predicted waypoint is relative to the same chunk origin:

$$\hat{T}_{t+h} = T_{\text{anchor},t}\Delta\hat{T}_{t,h}.$$

We do not add one waypoint's motion to the next within a chunk. At the next query, the previous K poses, excluding the current pose, are converted to coordinates relative to the new anchor. Gripper widths are absolute widths and need no coordinate conversion. Future GT actions and phase labels are never model inputs.

How are rollout errors computed? Table 6 reports *Episode-end FDE*: score the last valid predicted position of each episode, then average those errors over episodes. It does not average the endpoint errors of all chunks. We also report ADE, which averages position error over all valid waypoints within an episode before averaging over episodes. Let $E = 257$, let L_e be the number of scored waypoints in episode e, and let $d_{e,j} = 100\|\hat{p}_{e,j} - p_{e,j}\|_2$ be the world-position error in cm. Then

$$\text{ADE} = \frac{1}{E}\sum_e \frac{1}{L_e}\sum_{j=1}^{L_e} d_{e,j},$$

$$\text{Episode-end FDE} = \frac{1}{E}\sum_e d_{e,L_e}.$$

Each episode has equal weight, regardless of length. We compute these averages for each seed, then report mean and sample standard deviation over the three seeds.

Why do Tables 2 and 6 have different FDE values? Table 2 scores step 16 of every complete sliding window, whereas Table 6 scores only the episode's last valid point. For 240 of the 257 episodes, that point falls in a short final chunk. In GT/GT, the final chunk starts from the recorded pose, so a shorter remaining horizon can give a smaller error. Thus Table 6's 4.55 ± 0.14 cm is not a replacement for Table 2's 11.14 ± 0.29 cm. To compare feedback modes, use the matched GT/GT reference within Table 6.

For completeness, Chunk-FDE@16 over the 746 complete rollout chunks is 11.60 ± 0.32, 12.75 ± 0.42, and 16.79 ± 0.76 cm for BBox+Visual in GT/GT, Pred/GT, and Pred/Pred. Even these values need not exactly match Table 2: rollout queries do not overlap, and the revised loader corrects how RGB frames are matched to valid tracked poses. With that correction, the same episodes would yield 10,051 complete sliding windows instead of the original 10,024. We therefore recompute all three rollout modes with the same corrected alignment and query steps rather than mix the two evaluations.

Checks for a fair comparison. Initial prediction noise is fixed for each episode and query step across conditions and feedback modes. Checks confirm identical query/noise schedules, identical first-chunk predictions across modes, full coverage of the short final chunks, and no GT-pose resets in Pred/Pred. The nine rollout-specific tests and the full 59-test regression suite pass.

Which stages are affected? Table 7 groups waypoint errors by the recorded execution phase after prediction. For each phase, we first average waypoint errors within an episode, then give each episode equal weight. These are phase-wise ADE values, not the chunk endpoint FDE values in Table 4. In Pred/Pred, shuffling the object raises Grasp/Occlusion ADE from 12.11 to 19.98 cm. Shuffling the target raises Placement ADE from 21.48 to 37.91 cm and Episode-end FDE from 22.59 ± 0.78 to 41.93 ± 1.61 cm; shuffling the object gives Episode-end FDE of 24.09 ± 0.45 cm. Object cues therefore matter more during acquisition, while target cues matter more near placement. Even with correct prompts, episode-end S@10 and Combined@10 are only $16.86 \pm 1.37\%$ and $5.45 \pm 2.02\%$. These apply the usual endpoint checks to the episode's final valid point; they do not measure physical pick-and-place success.

What this experiment does not test. Every query still receives an image from the recorded video. A predicted action cannot change the next image, move an object, create contact, or end the episode. The predicted EE pose may therefore disagree with the camera viewpoint in the video. This experiment tests how prediction errors carry through history and pose anchors; it is not closed-loop robot control and cannot guarantee better or worse performance in deployment. It also uses the existing scenes and task, not new scenes or new manipulation skills.

C.4. Additional Information-Source Controls

No Prompt still receives the unmarked first frame and recorded action history. Removing both, so that only the current RGB frame remains, raises Chunk-FDE@16 by 2.40 cm and lowers S@10 by 3.98 percentage points (Table 8).

Table 7. Phase errors in Pred/Pred rollout with recorded RGB. Values are waypoint ADE (cm), averaged within each episode and phase, then across episodes; mean ± sample standard deviation over three seeds. Recorded phase labels are used only to group errors after prediction.

Condition	Approach	Pre-grasp	Grasp/occl.	Transport	Placement
BBox+Visual	6.97 ± 0.23	12.10 ± 0.45	12.11 ± 0.53	15.97 ± 0.59	21.48 ± 0.71
No Prompt	7.22 ± 0.12	13.18 ± 0.50	13.67 ± 0.60	19.14 ± 0.19	37.48 ± 0.29
Shuffled object	10.19 ± 0.21	19.33 ± 0.57	19.98 ± 0.72	22.15 ± 0.96	23.29 ± 0.34
Shuffled target	7.09 ± 0.29	12.18 ± 0.17	12.30 ± 0.08	19.10 ± 0.53	37.91 ± 1.41

Table 8. Seed-42 information-source control on the same 10,024 validation samples. No Prompt retains the unmarked first frame and $K = 4$ recorded action history, whereas Current RGB Only removes both task input and history. Grip. L1 is in millimeters.

Input	Pos. L2	FDE	Rot6D L2	Grip. L1	S@2/5/10 (%)
No prompt	9.12	17.39	0.2021	10.98	5.01/20.66/40.22
Current RGB only	10.84	19.79	0.2133	13.93	3.72/17.94/36.24

Table 9. Seed-42 observation-encoder control. Both rows retain the SPOT task encoder, prompt, $K = 4$ history, action head, split, and evaluation samples. The second row replaces only the current-frame observation encoder with the SigLIP tower used by π0.5. Translation errors and Grip. L1 are in centimeters.

Observation encoder	Pos. L2	FDE	Rot6D L2	Grip. L1	S@2/5/10 (%)
DINOv2 (default)	6.99	11.47	0.1965	1.13	3.35/20.83/52.80
π0.5 SigLIP	7.10	11.54	0.1969	1.33	3.93/20.81/50.96

These inputs therefore help together; this test does not tell us how much each contributes separately. Replacing only the current-image encoder with the π0.5 SigLIP tower barely changes position and rotation error but increases gripper error (Table 9). Changing the observation encoder alone therefore does not reproduce the external baseline's behavior.

C.5. External VLA Baseline Protocols

All external VLA rows use the same train/validation split, 16-step 10D action target, 10,024 validation samples, and training seeds 42/0/1. SmolVLA, OpenVLA-OFT, GR00T N1.6, and π0.5 use the current RGB frame, the same recorded 4-step action history as SPOT, and category-level language. They do not receive the first frame, rendered boxes, or box coordinates. Both object and target indices are removed from the instructions reported in Table 1, yielding prompts such as `pick fork and place to plate`.

For π0.5, only the action expert is fine-tuned. For SmolVLA, we train the action expert and state projection but freeze the vision/VLM backbone. OpenVLA-OFT uses rank-32 LoRA and its continuous OFT L1 action head. The models share data and action targets, but their task inputs and fine-tuning recipes differ. Table 1 therefore compares these tested systems, not architecture alone.

Table 10. Inference-time VLA language sensitivity. Indexed instructions retain both instance IDs; Object-ID removed retains the target ID; Category removes both IDs and corresponds to the protocol reported in Table 1. Values are three-seed means; checkpoints and non-language inputs are fixed within each model.

Model	Instruction	FDE (cm)	S@10 (%)
SmolVLA	Indexed	12.66	49.05
	Object-ID removed	13.98	42.68
	Category	16.85	37.21
OpenVLA-OFT	Indexed	11.11	56.68
	Object-ID removed	12.38	48.95
	Category	15.33	41.43
GR00T N1.6	Indexed	12.77	48.08
	Object-ID removed	15.42	35.38
	Category	17.39	32.42

Table 11. Seed-42 OpenVLA-OFT history control under indexed task language. The no-history model removes the 40D state/history projector and is trained and evaluated on the same sample windows. Grip. L1 is in millimeters. This diagnostic is separate from the category-language comparison in Table 1.

History	Pos. L2	FDE	Rot6D L2	Grip. L1	S@2/5/10 (%)
$K = 4$	6.16	11.01	0.1657	7.98	5.38/28.21/57.74
$K = 0$	6.68	11.33	0.1749	13.83	4.97/27.16/54.79

C.6. External VLA Input Diagnostics

Table 10 tests how much the language's object and target indices help. Removing the object index worsens all three VLAs; removing both indices worsens them further. An instruction such as `pick fork and place to plate` does not identify a particular fork or plate when several are visible. This loss of task information should not be interpreted as a defect in the VLA architecture. Table 11 tests

a different input: action history. Removing it has a modest effect on OpenVLA-OFT position error but raises Grip. L1 from 7.98 to 13.83 mm. That test uses language with instance indices, so it is separate from the main category-language comparison.

C.7. Five-Stage Phase Segmentation

What the five phases mean. Table 4 divides an episode into Approach (initial motion), Pre-grasp (slowing near the object), Grasp/Occlusion (gripper closing and initial lift), Transport (moving the grasped object), and Placement (approaching the target, releasing, and finishing). We find these boundaries from recorded gripper width and EE motion after prediction; the phase labels are not extra model inputs. "Occlusion" names the grasp interval, not an image-based visibility measurement.

Finding the grasp. We retain every third valid frame and median-filter the gripper width. The open width is the median over the first 12% of the episode, using at least five steps; the closed width is the episode's 10th percentile. Normalized openness is

$$q_t = (w_t - w_{\text{closed}})/(w_{\text{open}} - w_{\text{closed}}),$$

so values near zero indicate a closed gripper and values near one an open gripper. Between 8% and 78% of the episode, the first sustained run of two to five steps with $q_t \leq 0.25$ marks completed closure. Grasp starts just after the last $q_t \geq 0.75$ sample in the preceding 20% window. Pre-grasp starts roughly 10% of an episode earlier, adjusted to the lowest EE speed within a five-step neighborhood. Transport starts after closure and an initial-lift interval lasting 5% of the episode, with a minimum of two steps.

Finding release and placement. After 55% of the episode, the first two consecutive steps with $q_t \geq 0.40$ mark completed reopening. Release starts just after the last $q_t \leq 0.20$ sample in the preceding 20% window. If reopening is not sustained, we instead use the largest late increase in gripper width. Placement starts roughly 18% of an episode before release, adjusted to the strongest EE slowdown within a seven-step neighborhood. It therefore includes the final approach to the target, not just opening the gripper.

Fallback and scoring. If the episode is too short, the gripper moves by less than 15 mm, or closing/reopening cannot be found reliably, we use boundaries at 28%, 38%, 52%, and 78% of episode duration. The detector works for 217 of the 257 validation episodes; the remaining 40 use this fallback. We keep the boundaries in order and require at least two steps in each transition-centered phase. A manual check of 15 episodes across the scenes found no reversed phase order. Each waypoint error is grouped by that waypoint's phase. Chunk-FDE@16 and other endpoint criteria use the phase at step 16 of the prediction, not the phase of the input image.

Table 12. Pick/place endpoint results on the 183-episode high-confidence boundary subset. FDE is in centimeters and S@10 is in percent. The conclusions match the all-episode results in Table 5.

Split	Condition	Pick FDE	Place FDE	Pick S@10	Place S@10
Task half	No prompt	10.63	26.59	55.9	16.6
	Target only	10.72	**11.75**	53.5	**47.2**
	BBox + visual	**9.73**	13.68	**60.4**	38.7
Contact ± 15	No prompt	8.08	24.99	70.8	25.6
	Target only	9.42	**10.69**	61.9	**53.9**
	BBox + visual	**7.62**	13.19	**75.2**	43.8

C.8. Pick/Place Boundary Analysis

Table 5 uses a separate detector to estimate when pick and place contact occur. It also runs only after prediction, using recorded EE position and gripper width. We smooth both signals and find sustained closing and reopening. Near each transition, we select a low-speed point in the lowest part of the EE path, taking world Y as the upward axis. The highest EE position between the contacts is the transport apex. If the gripper signal is unreliable or places the contacts in the wrong order, we fall back to low points in the EE path alone.

We use these contacts in two ways. The *task-half* split labels a chunk Pick if its endpoint is before the transport apex, and Place otherwise. The *contact-window* split keeps only endpoints within ± 15 raw video frames (about ± 0.5 s) of a contact and assigns them to the nearer contact. Chunk-FDE@16 and S@2/5/10 score those selected endpoints. We average within each episode before averaging across episodes.

Detector confidence is higher when contacts form clear low points, EE speed is low, position and gripper signals agree, gripper motion is large enough, and pick precedes place. The full analysis uses all 257 validation episodes. Table 12 repeats it on the 183 episodes with high-confidence boundaries.

Both definitions give the same ranking: BBox+Visual is best for Pick FDE/S@10, Target Only is best for Place FDE/S@10, and No Prompt has the largest Pick–Place gap. This also holds on the high-confidence subset, so uncertain contact boundaries do not explain the main result.

D. Statistical Details for Prompting Baselines

Table 13 gives confidence intervals for the differences between prompting methods in Table 2. The paired bootstrap resamples seeds and episodes while keeping each method's results paired on the same data. An interval that excludes zero supports a difference under this resampling procedure; an interval that includes zero does not establish a clear difference.

Table 13. Hierarchical paired-bootstrap statistics for the prompting baselines, using 10,000 resamples over seeds and validation episodes. Each row reports BBox+Visual minus the named baseline with a 95% confidence interval. Success-rate differences are in percentage points (pp); bold intervals exclude zero.

Comparison	ΔPos. L2 (cm) [95% CI]	ΔFDE (cm) [95% CI]
vs. No prompt	-2.4905 **[$-2.8160, -2.1132$]**	-6.6132 **[$-7.3008, -5.8663$]**
vs. Target only	-0.1683 **[$-0.3084, -0.0150$]**	-0.4327 [$-0.8186, 0.0368$]
vs. Visual	$+0.0774$ [$-0.0699, 0.2009$]	-0.1097 [$-0.4149, 0.1794$]
vs. BBox	-0.2466 **[$-0.4217, -0.0560$]**	-0.7069 **[$-1.0822, -0.2920$]**

Comparison	ΔS@2 (pp) [95% CI]	ΔS@5 (pp) [95% CI]	ΔS@10 (pp) [95% CI]
vs. No prompt	-0.51 [$-1.61, 0.43$]	$+1.84$ **[$0.14, 3.34$]**	$+13.95$ **[$12.21, 15.65$]**
vs. Target only	$+0.46$ [$-0.05, 0.95$]	$+1.14$ **[$0.22, 2.07$]**	$+3.00$ **[$1.43, 4.53$]**
vs. Visual	-0.19 [$-0.70, 0.39$]	-1.35 **[$-2.25, -0.46$]**	$+0.20$ [$-2.33, 3.22$]
vs. BBox	$+0.81$ **[$0.16, 1.43$]**	$+2.29$ **[$1.16, 3.46$]**	$+4.12$ **[$2.56, 5.74$]**

Table 14. Prompt-noise robustness on all 10,024 validation samples. Combined values are offline endpoint criteria at 5/10 cm.

Condition	FDE (cm)	ΔFDE (%)	Rot. geo. (°)	Combined@5/10 (%)
Correct prompt	11.4655	–	9.95	15.81 / 38.67
Jitter 2%	11.6984	+2.03	10.20	15.65 / 38.06
Jitter 5%	12.3716	+7.90	10.27	14.66 / 35.01
Jitter 10%	14.2021	+23.87	10.89	11.40 / 27.86
Jitter 20%	17.5195	+52.80	11.28	9.13 / 24.95
Scale 0.8	**11.3393**	-1.10	**9.77**	**16.31 / 38.93**
Scale 0.9	11.4147	-0.44	9.84	15.89 / 38.84
Scale 1.1	11.5588	+0.81	10.02	15.73 / 37.95
Scale 1.2	11.6245	+1.39	10.20	15.17 / 36.88
Missing object	13.2412	+15.49	10.19	12.89 / 31.74
Missing target	18.2588	+59.25	10.78	12.51 / 29.14

E. Additional Prompt Analyses

Table 14 tests inaccurate or missing prompts. Larger shifts of the box center cause larger errors, while moderate changes in box size matter less. Removing the target is more harmful than removing the object.

With correct prompts, Chunk-FDE@16 is 11.47 cm. Jitter of 2% raises it by only 2.03%, but jitter of 5%, 10%, and 20% raises it by 7.90%, 23.87%, and 52.80%, respectively. Combined@10 falls from 38.67% to 38.06%, 35.01%, 27.86%, and 24.95%. Small box-center errors therefore have little effect, but large shifts reduce accuracy substantially.

Resizing boxes from $0.8\times$ to $1.2\times$ changes FDE by only -1.10% to $+1.39\%$, with Combined@10 between 36.88% and 38.93%. The slight improvement at $0.8\times$ comes from one seed and does not establish that smaller boxes are better. Together, the shift and resize tests suggest that box location matters more than exact box size. Removing the target raises FDE by 59.25% and lowers Combined@10 to 29.14%; removing the object raises FDE by 15.49% and gives Combined@10 of 31.74%. Rotation changes less: mean Rot. geo. stays between 9.77° and 11.28° across the tests. Incorrect prompts therefore affect where the trajectory goes more than how the EE is oriented.

Table 15 adds rotation, gripper, and combined endpoint checks. Object Only is close to No Prompt, while Target Only provides most of the endpoint improvement. Visual Prompt is best for rotation and Combined@5; BBox+Visual is best for Combined@10. Incorrect prompts change FDE and Combined success more than endpoint gripper-state accuracy. They mainly change the predicted destination rather than worsening every action component equally.

F. Additional Architecture Ablations

Interpreting the trajectory-head comparison. Flow matching improves the final waypoint, mean position error, orientation, and gripper prediction (Figure 4). Its benefit is therefore not limited to the chunk endpoint. Both heads use the same decoder architecture, so extra decoder capacity does not explain the gain. However, their training objectives and sampling procedures differ, and this experiment does not separate those effects. The reported errors also do not measure motion smoothness, prediction speed, or whether the trajectory is physically feasible.

What history contributes. At $K = 4$, history reduces gripper error more than FDE in relative terms. Recent motion may help distinguish stages that look similar in one image, but we did not directly test stage recognition. More history is not always better: $K = 12$ gives the lowest gripper error, while $K = 4$ gives better position accuracy than $K = 8/12$. We choose $K = 4$ because spatial prediction is the main objective. These tests use recorded history; Section C.3 separately tests errors in predicted history.

Table 15. Additional seed-42 task-oriented metrics on all 10,024 validation samples. Rot. geo. reports mean/endpoint geodesic error in degrees; R@10/20 is endpoint rotation success; G-end is endpoint gripper-state accuracy; and C@5/10 is combined offline endpoint success. Grip. L1 is in millimeters. Correct prompt variants use their corresponding trained checkpoints, whereas the shuffled and role-swap rows are inference-time interventions on the BBox+Visual checkpoint.

Condition	FDE (cm)	Rot. geo. mean/end (°)	Grip. L1 (mm)	R@10/20 (%)	G-end (%)	C@5/10 (%)
No prompt	17.39	10.10/16.30	10.98	36.69/71.29	80.34	16.02/29.94
Object only	17.28	10.26/16.18	10.96	36.95/72.15	81.09	15.60/31.57
Target only	11.44	10.47/16.79	12.05	31.70/72.40	79.77	15.01/34.88
Point prompt	11.38	9.94/16.21	**10.61**	34.61/73.57	**82.77**	16.05/37.01
BBox prompt	11.76	10.25/16.23	11.73	34.61/72.96	80.79	14.43/34.47
Visual prompt	11.75	**9.42/14.80**	10.95	**39.04/77.89**	80.87	**16.97**/37.29
BBox + visual	**11.47**	9.95/15.53	11.33	35.48/77.26	81.04	15.81/**38.67**
Shuffled object	13.84	10.90/17.58	11.48	30.18/71.43	80.56	12.68/29.93
Shuffled target	18.47	11.09/17.93	11.83	33.10/69.28	79.34	11.76/26.68
Role swap	22.08	11.23/17.85	11.59	30.94/68.86	79.62	9.86/24.77

Table 16. Ablation on visual encoder type. We compare frozen base-size visual encoders under the same SPOT configuration.

Split	Encoder	FDE (cm)	Pos. L2 (cm)	Rot. geo. (°)	Grip. L1 (mm)
All	PE-Base	15.44	8.91	10.91	10.85
	SigLIP-Base	12.04	7.24	**9.58**	**9.21**
	SAM-Base	17.59	9.29	10.93	9.28
	EVA2-Base	14.22	8.26	10.51	10.50
	DINOv2-Base	**11.47**	**6.99**	9.95	11.33
Scene 1	PE-Base	11.37	6.87	7.70	7.49
	SigLIP-Base	**10.83**	**6.64**	**7.38**	**6.74**
	SAM-Base	14.18	7.52	7.81	6.76
	EVA2-Base	10.96	6.65	7.64	7.28
	DINOv2-Base	11.23	7.00	7.67	6.93
Scene 2	PE-Base	18.88	10.19	12.42	9.67
	SigLIP-Base	12.48	7.02	**10.43**	**8.58**
	SAM-Base	19.98	10.01	12.23	10.17
	EVA2-Base	15.64	8.61	11.40	9.52
	DINOv2-Base	**11.16**	**6.32**	10.93	9.75
Scene 3	PE-Base	18.22	10.83	14.46	17.88
	SigLIP-Base	13.61	8.53	**12.25**	13.99
	SAM-Base	20.90	11.54	14.56	**12.54**
	EVA2-Base	17.96	10.55	14.21	17.00
	DINOv2-Base	**12.24**	**7.79**	12.58	20.48

Freezing and limits of the design choice. Freezing keeps the pretrained image features fixed while the prompt-fusion modules and trajectory decoder learn. This works better than full tuning in our experiment, but it does not prove that the tuned model overfits. Training/validation curves, different backbone learning settings, or more training data would help distinguish possible causes. We also change one component at a time, so separate improvements may not add up when combined. The encoder results below further show that the best model depends on which action component is measured.

No encoder is best on every metric (Table 16). DINOv2 has the lowest overall FDE/Pos. L2 (11.47/6.99 cm), while SigLIP has the lowest Rot. geo./Grip. L1 (9.58°/9.21 mm). SAM has the highest FDE but nearly matches SigLIP's gripper error, again showing that good gripper prediction does not guarantee good position prediction. Encoder choice matters more in the harder scene: the gap between highest and lowest FDE grows from 3.35 cm in Scene 1 to 8.66 cm in Scene 3. DINOv2 is best for position in Scenes 2/3, while SigLIP is best for their rotation. We choose DINOv2 for position accuracy, not because it is best for every output. These differences may reflect what each visual pretraining objective emphasizes, but this experiment does not identify the cause.

Table 17. Ablation on DINOv2 encoder size. Translation errors are in centimeters and Grip. L1 is in millimeters; lower is better.

Split	Encoder	FDE (cm)	Pos. L2 (cm)	Rot. geo. (°)	Grip. L1 (mm)
All	DINOv2-Small	**11.29**	7.01	10.70	13.86
	DINOv2-Base	11.47	6.99	9.95	11.33
	DINOv2-Large	11.35	**6.79**	**9.39**	**10.81**
Scene 1	DINOv2-Small	10.85	6.85	8.15	7.19
	DINOv2-Base	11.23	7.00	7.67	6.93
	DINOv2-Large	**9.88**	**6.09**	**7.18**	**6.76**
Scene 2	DINOv2-Small	**10.57**	**6.17**	11.48	**8.85**
	DINOv2-Base	11.16	6.32	10.93	9.75
	DINOv2-Large	12.44	6.79	**10.45**	9.78
Scene 3	DINOv2-Small	12.82	8.29	14.04	30.91
	DINOv2-Base	**12.24**	**7.79**	12.58	20.48
	DINOv2-Large	12.51	7.96	**11.81**	**18.71**

G. Additional Ablation: Vision Encoder Size

We further compare DINOv2 backbones of different sizes under the same SPOT setting. All variants use the combined visual and coordinate bounding-box prompt, frozen visual features, flow matching, and the same scene-aware evaluation protocol. Table 17 reports the results.

A larger DINOv2 does not consistently improve endpoint position accuracy. Overall FDE differs by only 0.18 cm

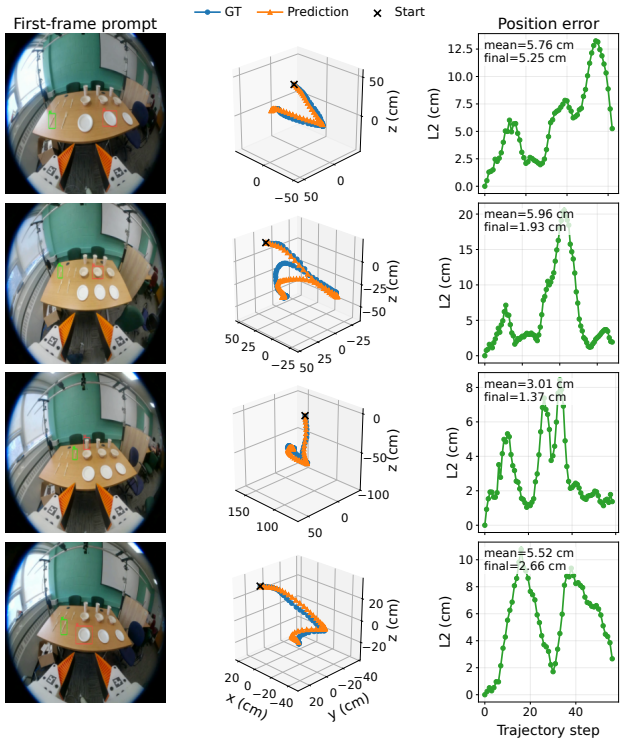

Figure 12. **Additional full-trajectory visualizations for Scene 1.** Stitched trajectories and position errors are reported in centimeters.

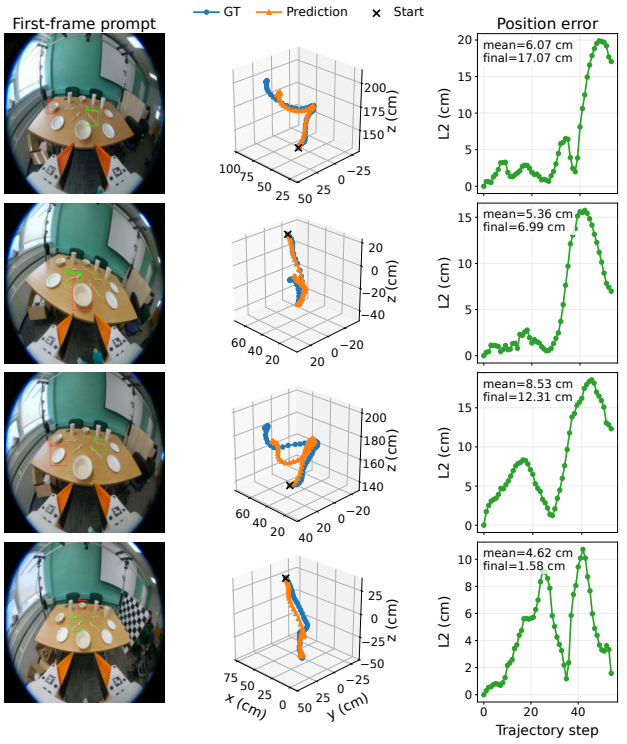

Figure 13. **Additional full-trajectory visualizations for Scene 2.** Stitched trajectories and position errors are reported in centimeters.

across sizes, with Small slightly best. Large improves rotation and gripper error over Base by 0.56° and 0.52 mm, but its Scene-2 FDE is 1.87 cm worse than Small's. In Scene 3, Large reduces gripper error by 12.20 mm relative to Small, while Base still has the lowest FDE/Pos. L2. More capacity therefore helps some outputs and scenes but not all. We retain Base as a balance of position accuracy, rotation/gripper accuracy, and compute, using the same size for every scene.

H. More Visualization Results

We provide additional full-episode trajectories from the three scenes in Figs. 12, 13, and 14, as well as further first-chunk examples in Fig. 15.

I. Limitations

Dataset scope. EgoSPT covers a single manipulated-object category (forks) and a single task (pick-and-place) across three scenes. Five fork instances and nine receptacle targets introduce spatial ambiguity, while the scenes vary layout and clutter within this task family. These variations do not establish transfer to other object categories or manipulation skills. Accordingly, the measured prompt benefits and the phase-specific effects of object and target cues should be interpreted within this setting. Whether these findings extend to objects with different geometries or grasp affordances, deformable items, articulated manipulation, or multi-step tasks requires additional data and evaluation.

Evaluation scope. We evaluate predictions from recorded data, not a robot acting in the world. Section 5.6 shows that errors accumulate when predicted states are fed back. Prompts reduce this drift but do not remove it. Because later images still come from the recording, they cannot show the consequences of a wrong action, such as a missed grasp or displaced object. Predicted poses may also disagree with the recorded camera view. Passing an offline endpoint check is therefore not evidence of completing the physical task; real-time closed-loop execution remains untested.

The method also assumes that the first-frame object and target prompts are available and reasonably accurate. Strong prompt noise, severe occlusion, or target regions that leave the camera view may reduce performance. Finally, while frozen foundation visual encoders perform better in our experiments, their behavior may depend on the visual domain, camera viewpoint, and object categories.

J. Ethics and Broader Impact

EgoSPT is collected and annotated by trained experts rather than crowdsourced workers. The videos are captured from an egocentric device during table-top manipulation and are intended to avoid collecting identifiable personal information.

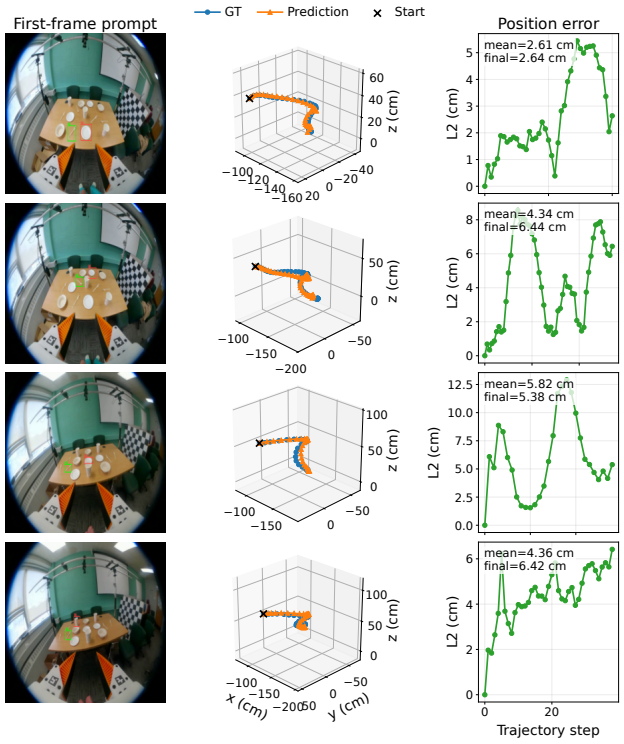

Figure 14. **Additional full-trajectory visualizations for Scene 3.** Stitched trajectories and position errors are reported in centimeters.

The dataset is designed for research on visually grounded manipulation and does not involve high-risk generated media, scraped web data, or personal decision making.

Spatial prompting can make robot task specification more direct and accessible, especially in cluttered scenes where language instructions are ambiguous. At the same time, improved manipulation policies may be unsafe if deployed without closed-loop monitoring, collision checking, or task-level constraints. Any deployment should therefore include safety checks, workspace limits, and human supervision appropriate to the robot platform and environment.

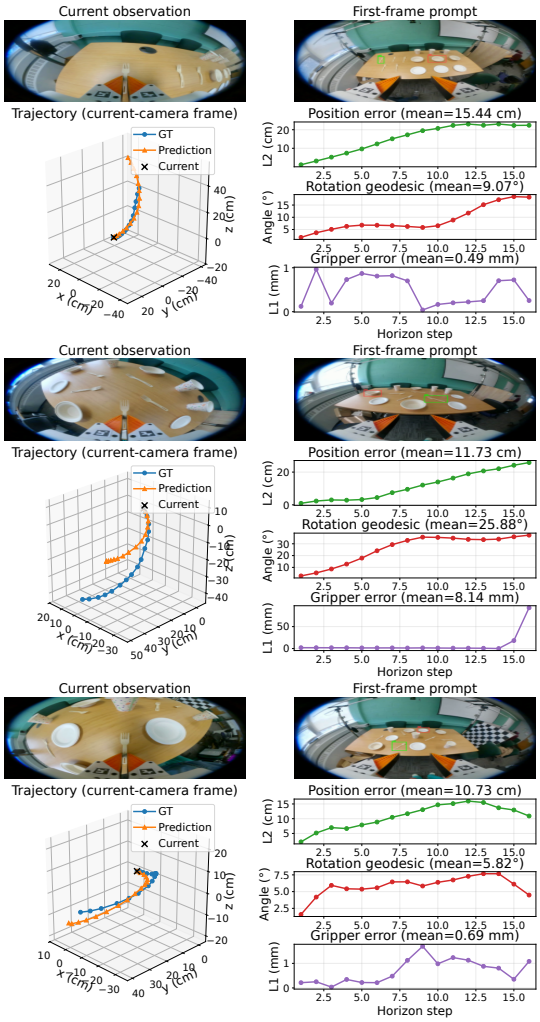

Figure 15. **Additional first-chunk visualizations for three scenes.** Scenes 1–3 are shown top to bottom; units are cm (trajectory/position), degrees (rotation geodesic), and mm (gripper width).